

Τεχνικό Συνοδευτικό Εγχειρίδιο: από το WISDM στο Κατανεμημένο ML

Εσωτερική λειτουργία της προεπεξεργασίας, του PySpark MLlib και της εκτέλεσης στο Google Cloud

Κώστας Φύκατας
Ακαδημαϊκό έτος 2025–2026

Περιεχόμενα

Ενότητες 1–15	Ενότητες 16–28
1. Τα δεδομένα WISDM	16. Ένα μοντέλο με κατανεμημένους υπολογισμούς
2. Πού αποθηκεύονται τα δεδομένα	17. Από την περιγραφή των πράξεων στην εκτέλεση
3. Πώς το Spark διαβάζει τα δεδομένα	18. Γιατί το shuffle έχει κόστος
4. Έλεγχος και καθαρισμός των αρχικών μετρήσεων	19. Προσωρινή μνήμη και αποθήκευση στον δίσκο
5. Χρονικά παράθυρα	20. Τοπική εκτέλεση και Google Cloud
6. Η διαφορά ανάμεσα σε partition και χρονικό παράθυρο	21. Αποτελέσματα της εκτέλεσης στο Google Cloud
7. Υπολογισμός των χαρακτηριστικών	22. Μετρικές αξιολόγησης με παραδείγματα
8. Έλεγχος των παραθύρων και των χαρακτηριστικών	23. Η διαφορά από τη βαθιά μάθηση
9. Συνένωση επιταχυνσιομέτρου και γυροσκοπίου	24. PySpark ML έναντι scikit-learn
10. Τι περιγράφει μία εγγραφή του μοντέλου	25. Παραδείγμα από τη μέτρηση έως την πρόβλεψη
11. Διάνυσμα χαρακτηριστικών, τυποποίηση και ετικέτες	26. Πού βρίσκεται κάθε στάδιο στον κώδικα
12. Διαχωρισμός εκπαίδευσης, επικύρωσης και δοκιμής	27. Επιλογές σχεδιασμού και περιορισμοί
13. Τι λαμβάνει η εκπαίδευση του μοντέλου	28. Συμπέρασμα
14. Πολυταξική Λογιστική Παλινδρόμηση	
15. Random Forest	

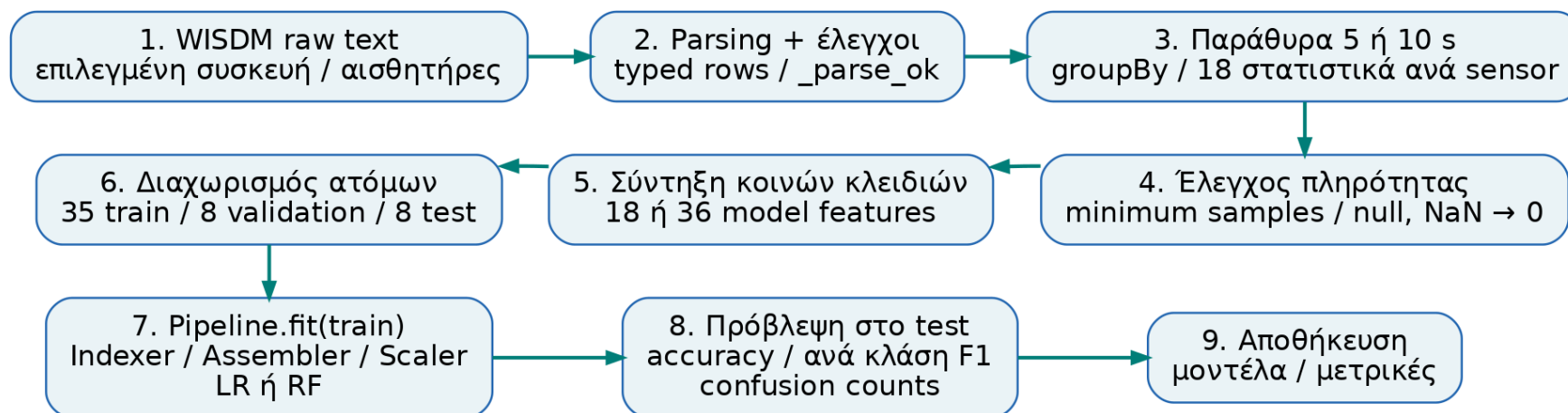
Σκοπός του εγχειριδίου

Το τεχνικό εγχειρίδιο εξηγεί πώς λειτουργεί η εφαρμογή har-spark-cloud, ακολουθώντας τη διαδρομή από μια γραμμή μετρήσεων του WISDM έως την πρόβλεψη μιας δραστηριότητας. Συμπληρώνει την κύρια αναφορά με περισσότερες λεπτομέρειες για τα δεδομένα, τον κώδικα και την εκτέλεση του Spark.

Τα παραδείγματα βασίζονται στην υλοποίηση του έργου. Το `src/har_spark/ingest.py` περιέχει την ανάγνωση και τους ελέγχους εισόδου. Το `features.py` δημιουργεί τα παράθυρα και τα χαρακτηριστικά, ενώ το `model.py` περιέχει τα μοντέλα και τις μετρικές. Το `pipeline.py` συνδέει τα στάδια και το `spark.py` δημιουργεί το τοπικό περιβάλλον Spark. Για το νέφος χρησιμοποιείται το `cloud/cloud_pipeline.py`.

Οι βασικές ρυθμίσεις βρίσκονται στο `configs/watch_fused_10s.yaml`. Τα αποτελέσματα του κύριου πειράματος προέρχονται από το `results/20260827T054435Z/`, ενώ η εκτέλεση στο νέφος τεκμηριώνεται στα `cloud/evidence/` και `cloud/deployment.md`. Όπου χρησιμοποιούνται υποθετικοί αριθμοί για την εξήγηση ενός βήματος, αυτό αναφέρεται στο παράδειγμα. Οι ερμηνείες των αποτελεσμάτων διακρίνονται από τις καταγεγραμμένες μετρήσεις.

Πριν από τις λεπτομέρειες, η βασική διαδρομή είναι η εξής. Μια γραμμή περιγράφει μία μέτρηση ενός αισθητήρα. Οι μετρήσεις του ίδιου χρονικού παραθύρου συγκεντρώνονται και δίνουν 18 αριθμητικά χαρακτηριστικά. Τα χαρακτηριστικά των δύο αισθητήρων ενώνονται σε μία εγγραφή με 36 αριθμούς. Στην εκπαίδευση, η εγγραφή συνοδεύεται από τη γνωστή δραστηριότητα· στην πρόβλεψη, το μοντέλο χρησιμοποιεί τους 36 αριθμούς για να εκτιμήσει τη δραστηριότητα. Έτσι αλλάζει σταδιακά αυτό που περιγράφει μια εγγραφή, από στιγμιαία μέτρηση σε παράδειγμα κίνησης. Η ενότητα 25 ακολουθεί αυτή τη διαδρομή με συγκεκριμένες γραμμές δεδομένων.



Σχήμα 1. Ροή δεδομένων και μοντέλων. Πηγή: `ingest.py`, `features.py`, `model.py`, `pipeline.py`.

1. Τα δεδομένα WISDM

1.1 Τι καταγράφουν οι αισθητήρες

Το **WISDM Smartphone and Smartwatch Activity and Biometrics Dataset** περιέχει δεδομένα 51 συμμετεχόντων. Κάθε άτομο εκτέλεσε 18 δραστηριότητες για περίπου τρία λεπτά την καθεμία. Ο επιδιωκόμενος ρυθμός δειγματοληψίας ήταν 20 Hz, δηλαδή περίπου μία μέτρηση ανά 50 ms. Καταγράφηκαν τέσσερις ροές αισθητήρων:

Συσκευή	Αισθητήρας	Τι εκφράζουν οι τρεις άξονες
τηλέφωνο	επιταχυνσιόμετρο	γραμμική επιτάχυνση μαζί με την επίδραση της βαρύτητας, σε m/s ²
τηλέφωνο	γυροσκόπιο	γωνιακή ταχύτητα περιστροφής, σε rad/s
ρολόι	επιταχυνσιόμετρο	επιτάχυνση στον καρπό, σε m/s ²
ρολόι	γυροσκόπιο	περιστροφή του καρπού, σε rad/s

Το επιταχυνσιόμετρο καταγράφει τη γραμμική επιτάχυνση, μαζί με την επίδραση της βαρύτητας. Το γυροσκόπιο καταγράφει πόσο γρήγορα περιστρέφεται η συσκευή γύρω από κάθε άξονα. Οι δύο αισθητήρες συμπληρώνουν ο ένας τον άλλο: κινήσεις με παρόμοια επιτάχυνση μπορεί να διαφέρουν ως προς την περιστροφή.

Οι μετρήσεις δεν περιλαμβάνουν GPS ή γεωγραφική θέση. Κάθε εγγραφή περιέχει τον κωδικό συμμετέχοντα, τη δραστηριότητα, τον χρόνο και τις τιμές των τριών αξόνων του αισθητήρα.

1.2 Μορφή μιας ακατέργαστης γραμμής

Κάθε γραμμή του αρχείου έχει την ακόλουθη μορφή:

`subject_id, activity, timestamp, x, y, z;`

Για παράδειγμα, στο αρχείο επιταχυνσιόμετρου του συμμετέχοντα 1600 υπάρχουν οι μετρήσεις:

`1600,A,90426708196641,7.091625,-0.5916671,8.195502;`

`1600,A,90426757696641,4.972757,-0.15831658,6.6967316;`

`1600,A,90426807196641,3.25372,-0.19183542,6.107758;`

Προέρχονται από το `raw/watch/accel/data_1600_accel_watch.txt`. Η πρώτη γραμμή αναλύεται ως εξής:

Πεδίο	Παράδειγμα	Τύπος που επιβάλλει ο κώδικας	Έννοια
subject_id	1600	integer	ψευδωνυμοποιημένο ID συμμετέχοντα
activity	A	string	κωδικός της δραστηριότητας· A = walking
timestamp_ns	90426708196641	long	χρονική ένδειξη που η εφαρμογή χειρίζεται σε ns
x	7.091625	double	τιμή άξονα x
y	-0.5916671	double	τιμή άξονα y
z	8.195502	double	τιμή άξονα z· ακολουθεί ; στο τέλος της γραμμής

Η εφαρμογή χρησιμοποιεί το timestamp ως ακέραια χρονική ένδειξη σε νανοδευτερόλεπτα, όχι ως ημερομηνία. Η διαφορά διαδοχικών τιμών είναι περίπου 50.000.000 ns, και η διάρκεια του παραθύρου μετατρέπεται στην ίδια μονάδα με `seconds × 1_000_000_000`.

Μια αντίστοιχη γραμμή από το γυροσκόπιο είναι:
1600, A, 90426757696641, 0.3149441, -1.0222765, -0.3099616;
Τα ονόματα x, y, z είναι ίδια, αλλά οι μονάδες διαφέρουν: στο γυροσκόπιο είναι rad/s, ενώ στο επιταχυνσιόμετρο m/s².

1.3 Κωδικοί δραστηριοτήτων

Οι ετικέτες των δραστηριοτήτων είναι γράμματα. Η αντιστοίχισή τους στο WISDM είναι:

Κωδικός	Δραστηριότητα	Κωδικός	Δραστηριότητα
A	walking	J	eating pasta
B	jogging	K	drinking
C	stairs	L	eating sandwich
D	sitting	M	kicking
E	standing	O	playing catch
F	typing	P	dribbling
G	brushing teeth	Q	writing
H	eating soup	R	clapping
I	eating chips	S	folding clothes

Δεν υπάρχει δραστηριότητα με κωδικό N. Για αυτό ο έλεγχος του κώδικα συνδυάζει τη συνθήκη `^[A-S]$` με την εξαίρεση `activity != "N"`.

1.4 Πόσες αρχικές μετρήσεις χρησιμοποιούνται

Το κύριο πείραμα χρησιμοποιεί τους δύο αισθητήρες του ρολογιού. Η καταμέτρηση των τοπικών αρχείων, πριν από το φιλτράρισμα, έδωσε:

Ροή αισθητήρα	Αρχικές γραμμές
επιταχυνσιόμετρο ρολογιού	3.777.046
γυροσκόπιο ρολογιού	3.440.342
σύνολο	7.217.388

Ο ρυθμός των 20 Hz δεν εξασφαλίζει ότι κάθε παράθυρο 10 s περιέχει ακριβώς 200 μετρήσεις. Για αυτό η εφαρμογή δέχεται ένα ελάχιστο πλήθος μετρήσεων ανά παράθυρο, αντί να απαιτεί ακριβώς το θεωρητικό πλήθος.

2. Πού αποθηκεύονται τα δεδομένα

2.1 Τοπική αποθήκευση

Το αρχικό συμπιεσμένο αρχείο διατηρείται στο:

```
data/raw/wisdm-dataset.zip
```

Μετά την αποσυμπίεση, τα αρχεία οργανώνονται ανά συσκευή και αισθητήρα:

```
data/raw/wisdm-dataset/raw/
```

```
├── phone/accel/*.txt
├── phone/gyro/*.txt
├── watch/accel/*.txt
└── watch/gyro/*.txt
```

Το αρχείο ρυθμίσεων του κύριου πειράματος επιλέγει το ρολόι, τους δύο αισθητήρες και παράθυρα 10 s:

```
raw_glob: "data/raw/wisdm-dataset/raw/watch/*/*.txt"
```

```
device: watch
```

```
sensors: [accel, gyro]
```

```
window_seconds: 10
```

```
minimum_samples_per_window: 150
```

Τα χαρακτηριστικά που προκύπτουν μπορούν να αποθηκευτούν στο data/processed/watch_accel_gyro_10s.parquet. Το Parquet είναι μορφή αποθήκευσης ανά στήλη, με σχήμα δεδομένων και συμπίεση. Επιτρέπει να επαναληφθεί η εκπαίδευση από τα έτοιμα χαρακτηριστικά, χωρίς νέα επεξεργασία των αρχικών αρχείων. Τα αρχεία κειμένου παραμένουν ανέπαφα.

2.2 Αποθήκευση στο Google Cloud

Στην εκτέλεση στο Google Cloud χρησιμοποιήθηκε το ιδιωτικό bucket:

```
gs://har-wisdm-student-2026-data/
```

```
├── code/
├── raw/watch/accel/*.txt
├── raw/watch/gyro/*.txt
└── results/cloud-watch-10s-20260827-055742/
```

Η εφαρμογή διαβάζει τα αρχεία με το μοτίβο διαδρομής gs://.../raw/watch/*/*.txt και αποθηκεύει τα χαρακτηριστικά Parquet, τα μοντέλα και τα αποτελέσματα JSON σε ξεχωριστή διαδρομή για κάθε εκτέλεση.

Στα στοιχεία της εγκατάστασης καταγράφονται ενεργά τα Uniform Bucket-Level Access και Public Access Prevention. Πρόκειται για την κατάσταση της συγκεκριμένης εκτέλεσης, όχι για έλεγχο της σημερινής κατάστασης του λογαριασμού.

2.3 Πώς διατηρούνται ανέπαφα τα αρχικά αρχεία

Οι πράξεις select, withColumn, where, groupBy και join περιγράφουν νέα DataFrames. Δεν τροποποιούν το αρχείο από το οποίο διαβάστηκαν οι μετρήσεις. Η spark.read.text(path) κάνει την ανάγνωση, ενώ η αποθήκευση αποτελεσμάτων ζητείται χωριστά με .write....

Η εφαρμογή χρησιμοποιεί επίσης `mode("errorifexists")`: αν υπάρχει ήδη ο φάκελος εξόδου, η εγγραφή αποτυγχάνει αντί να αντικαταστήσει τα περιεχόμενά του. Έτσι ξεχωρίζουν τα αρχικά αρχεία, η προσωρινή επεξεργασία και τα νέα αποτελέσματα. Το cache αφορά την προσωρινή διατήρηση υπολογισμένων δεδομένων· τα Parquet, τα μοντέλα και τα JSON είναι τα αρχεία που αποθηκεύονται για μετέπειτα χρήση.

3. Πώς το Spark διαβάζει τα δεδομένα

3.1 SparkSession

Η εφαρμογή επικοινωνεί με το Spark μέσω ενός `SparkSession`. Από αυτό ξεκινούν η ανάγνωση δεδομένων, οι πράξεις στα `DataFrames` και η χρήση των μοντέλων. Τοπικά, το `build_spark()` διαμορφώνει τη συνεδρία ως εξής:

```
builder = (  
    SparkSession.builder.appName(str(settings["app_name"]))  
    .config("spark.sql.shuffle.partitions", str(settings["shuffle_partitions"]))  
    .config("spark.driver.memory", str(settings["driver_memory"]))  
    .config("spark.sql.session.timeZone", "UTC")  
)
```

Το `appName` δίνει όνομα στην εφαρμογή, ώστε να αναγνωρίζεται στα αρχεία καταγραφής και στο Spark UI. Οι επόμενες ρυθμίσεις ορίζουν 8 partitions για τα τοπικά SQL shuffle στάδια, 4 GiB μνήμης για τον driver και ζώνη ώρας UTC. Στη συνέχεια, το `master="local[6]"` επιτρέπει έως έξι νήματα εκτέλεσης στην ίδια μηχανή.

Στο πρόγραμμα εκκίνησης του cloud δεν ορίζεται `.master(...)`, επειδή το περιβάλλον Spark παρέχεται από τη διαχειριζόμενη υπηρεσία.

3.2 Από τις γραμμές κειμένου στις στήλες

Η ανάγνωση ξεκινά με την εντολή:

```
raw = spark.read.text(path).withColumn("source_file", F.input_file_name())
```

Το `spark.read.text` τοποθετεί κάθε γραμμή κειμένου στη στήλη `value`. Η πρόσθετη στήλη `source_file` κρατά τη διαδρομή προέλευσης. Από αυτή αναγνωρίζονται η συσκευή (phone ή watch) και ο αισθητήρας (accel ή gyro).

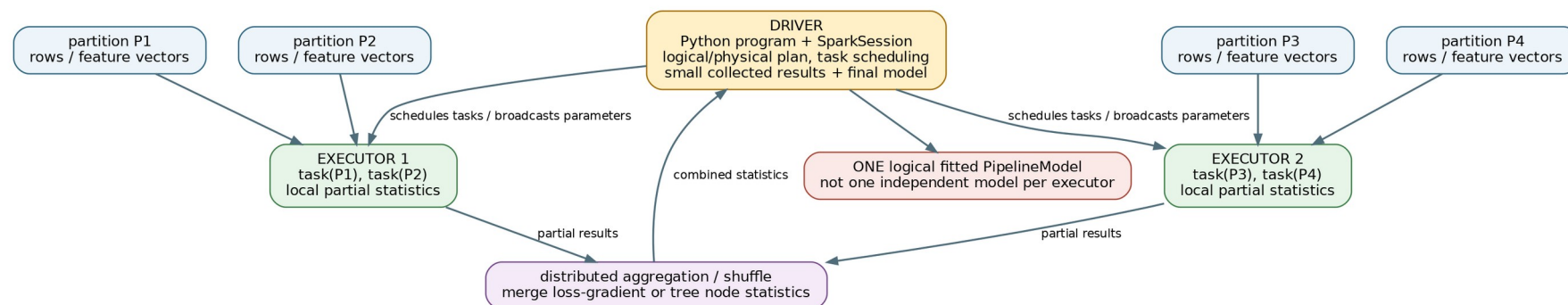
Μετά την ανάλυση των πεδίων, το DataFrame έχει τις ακόλουθες στήλες:

Στήλη	Τύπος Spark	Ρόλος
raw_line	string	αρχική γραμμή για έλεγχο σφαλμάτων
source_file	string	αρχείο προέλευσης
device	string	phone/watch/unknown
sensor	string	accel/gyro/unknown
field_count	integer	πλήθος πεδίων που χωρίζονται με κόμμα
subject_id	integer	κωδικός συμμετέχοντα
activity	string	κωδικός δραστηριότητας
timestamp_ns	long	χρονική ένδειξη
x,y,z	double	τιμές αξόνων αισθητήρα
_parse_ok	boolean	αποτέλεσμα των ελέγχων εγκυρότητας

3.3 Λογικό DataFrame και φυσικά partitions

Στον κώδικα, το DataFrame εμφανίζεται ως ένας πίνακας. Κατά την εκτέλεση, όμως, οι εγγραφές του χωρίζονται σε partitions. Κάθε επιμέρους εργασία, ή task, επεξεργάζεται συνήθως ένα partition. Σε ένα cluster, διαφορετικοί executors μπορούν να εκτελούν τέτοιες εργασίες παράλληλα.

Το πλήθος των αρχικών partitions εξαρτάται από τα αρχεία και τις ρυθμίσεις ανάγνωσης. Δεν καταγράφηκε ως σταθερός αριθμός στο πείραμα. Η τοπική τιμή `shuffle_partitions`: 8 αφορά τα SQL shuffle στάδια, όχι απαραίτητα την αρχική ανάγνωση. Αντίστοιχα, οι 2–4 executors του cloud δεν σημαίνουν 2–4 partitions, αφού κάθε executor μπορεί να εκτελέσει πολλά tasks.

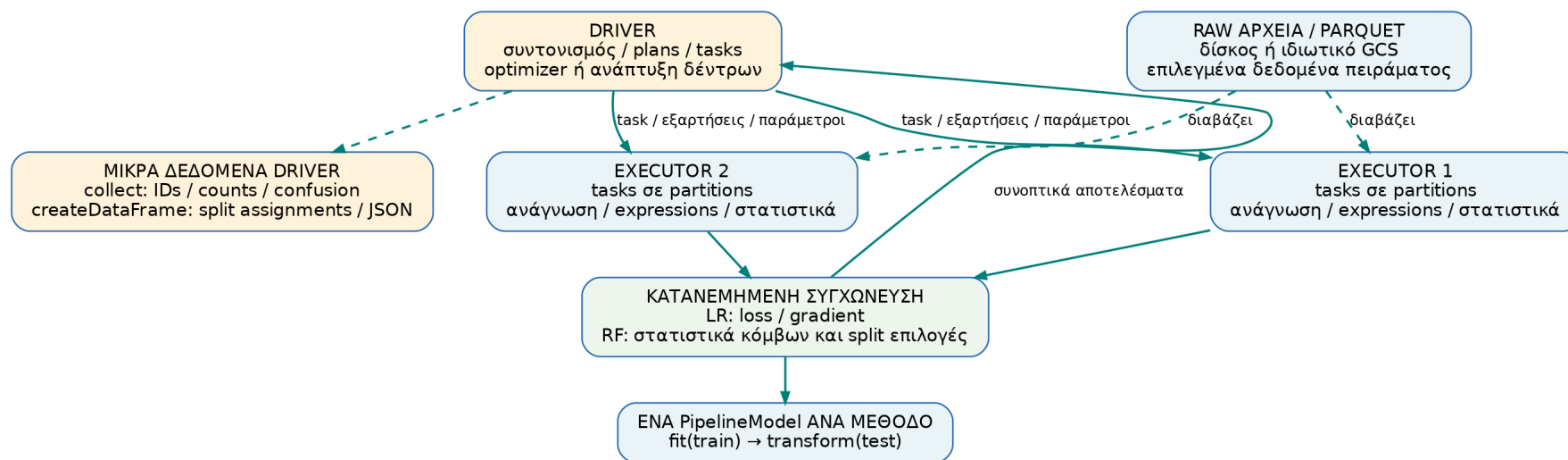


Σχήμα 2. Ενδεικτική κατανομημένη εκτέλεση. Το πλήθος partitions στο σχήμα δεν αποτελεί μέτρηση του run.

Ο driver οργανώνει την εκτέλεση και στέλνει τις εργασίες με τις απαραίτητες παραμέτρους στους executors. Οι εργασίες ανάγνωσης προσπελαίνουν τα αρχεία στον δίσκο ή στο Cloud Storage. Αρχικά ένα partition μπορεί να περιγράφει ένα αρχείο ή εύρος bytes του· μετά από shuffle, τα δεδομένα έχουν ανακαταταξινομηθεί.

Ο driver δεν συγκεντρώνει ολόκληρο το αρχικό σύνολο δεδομένων. Λαμβάνει όμως μικρά αποτελέσματα μέσω collect, όπως τους κωδικούς συμμετεχόντων, καταμετρήσεις και τα κελιά του πίνακα σύγχυσης. Δημιουργεί επίσης τον μικρό πίνακα που αντιστοιχίζει συμμετέχοντες στα σύνολα εκπαίδευσης, επικύρωσης και δοκιμής. Στο cloud δημιουργεί ακόμη ένα μικρό DataFrame για την εγγραφή JSON.

Η διανομή δεδομένων εξαρτάται και από τον αλγόριθμο: η Λογιστική Παλινδρόμηση μεταδίδει τους τρέχοντες συντελεστές με broadcast, δηλαδή διαθέτει ένα αντίγραφο τους σε κάθε executor που τους χρειάζεται. Η εφαρμογή δεν κάνει ξεχωριστό ρητό broadcast των σταθερών του παραθύρου.



Σχήμα 3. Ανάγνωση, συντονισμός και μικρά αποτελέσματα. Πηγή: ο κώδικας εφαρμογής και οι Spark ML αλγόριθμοι.

3.4 Driver, executor και task

Οι τέσσερις βασικοί όροι περιγράφουν διαφορετικούς ρόλους:

- Ο **driver** εκτελεί το κύριο πρόγραμμα, οργανώνει τα στάδια και συγκεντρώνει μικρά τελικά αποτελέσματα.
- Ο **executor** είναι διεργασία που εκτελεί tasks και διατηρεί προσωρινά δεδομένα για επαναχρησιμοποίηση.
- Το **task** είναι μια επιμέρους εργασία ενός σταδίου πάνω σε ένα partition.
- Το **partition** είναι το τμήμα δεδομένων που επεξεργάζεται αυτή η εργασία.

Στην εφαρμογή, το collect() χρησιμοποιείται για μικρούς πίνακες: καταμετρήσεις ποιότητας, τους 51 διαφορετικούς κωδικούς συμμετεχόντων και έως 18×18 κελιά σύγχυσης. Οι εκατομμύρια αρχικές μετρήσεις παραμένουν καταταξινομημένες.

4. Έλεγχος και καθαρισμός των αρχικών μετρήσεων

4.1 Ανάλυση μιας γραμμής βήμα προς βήμα

Η εφαρμογή αφαιρεί πρώτα τα περιττά κενά και το τελικό ;. Στη συνέχεια χωρίζει τη γραμμή στα κόμματα και μετατρέπει τα πεδία στους κατάλληλους τύπους:

```
cleaned = F.regexp_replace(F.trim(F.col("value")), r";\s*$", "")
parts = F.split(cleaned, ",")
```

```
F.size("parts").alias("field_count"),
F.col("parts").getItem(0).cast("int").alias("subject_id"),
F.trim(F.col("parts").getItem(1)).alias("activity"),
F.col("parts").getItem(2).cast("long").alias("timestamp_ns"),
F.col("parts").getItem(3).cast("double").alias("x"),
F.col("parts").getItem(4).cast("double").alias("y"),
F.col("parts").getItem(5).cast("double").alias("z"),
```

Το `size(parts)` επιτρέπει να ελεγχθεί ότι υπάρχουν ακριβώς έξι πεδία. Ο κωδικός συμμετέχοντα γίνεται `int`, ο χρόνος `long` και οι τρεις άξονες `double`. Στο πεδίο της δραστηριότητας εφαρμόζεται επιπλέον `trim` για την αφαίρεση κενών.

Με τις μη-ANSI ρυθμίσεις του πειράματος, μια αποτυχημένη αριθμητική μετατροπή δίνει `null`, ώστε η γραμμή να απορριφθεί από τον επόμενο έλεγχο. Σε ANSI mode, η ίδια είσοδος μπορεί να προκαλέσει σφάλμα.

Η συσκευή και ο αισθητήρας αναγνωρίζονται από τη διαδρομή του αρχείου, αφού μετατραπεί σε πεζά γράμματα. Για τη συσκευή χρησιμοποιείται:

```
F.when(source.contains("/phone/"), F.lit("phone"))
  .when(source.contains("/watch/"), F.lit("watch"))
  .otherwise(F.lit("unknown"))
```

Αντίστοιχος έλεγχος των `/accel/` και `/gyro/` αναγνωρίζει τον αισθητήρα. Μια μη αναγνωρισμένη διαδρομή παίρνει τιμή `unknown`.

4.2 Πότε μια γραμμή περνά τον έλεγχο

Η στήλη `_parse_ok` είναι αληθής μόνο όταν ικανοποιούνται όλες οι παρακάτω συνθήκες:

```
(F.col("field_count") == 6)
& valid_activity
& required_not_null
& (F.col("timestamp_ns") >= 0)
& F.col("device").isin("phone", "watch")
& F.col("sensor").isin("accel", "gyro")
```

Κάθε συνθήκη ελέγχει μια διαφορετική πλευρά της εγγραφής:

Έλεγχος	Τι απορρίπτει
ακριβώς 6 πεδία	ελλιπή ή επιπλέον πεδία στη γραμμή
activity <code>^[A-S]\$</code> και όχι N	μη επιτρεπτό κωδικό δραστηριότητας
μη-null subject/timestamp/x/y/z	αποτυχημένες μετατροπές ή ελλείπουσες απαραίτητες τιμές
timestamp ≥ 0	αρνητική χρονική ένδειξη
device phone/watch	μη αναγνωρισμένη διαδρομή συσκευής
sensor accel/gyro	μη αναγνωρισμένη διαδρομή αισθητήρα

Το `_parse_ok` διατηρείται προσωρινά για την καταμέτρηση έγκυρων και μη έγκυρων γραμμών με `groupBy("_parse_ok").count()`. Έπειτα αφαιρούνται οι γραμμές που απέτυχαν.

Ο έλεγχος `isNotNull()` δεν αποκλείει από μόνος του τιμές NaN ή άπειρο. Δεν υπάρχει ξεχωριστός έλεγχος ότι όλες οι τιμές των αξόνων είναι πεπερασμένες. Αυτό είναι όριο του ελέγχου εισόδου, όχι ένδειξη ότι τέτοιες τιμές εντοπίστηκαν στα αρχεία.

4.3 Επιλογή των δεδομένων του πειράματος

Αφού ελεγχθεί η μορφή των γραμμών, η `valid_records()` κρατά μόνο τις μετρήσεις από τη συσκευή και τους αισθητήρες που έχουν επιλεγεί στις ρυθμίσεις του πειράματος:

```
parsed.where(  
    F.col("_parse_ok")  
    & (F.col("device") == F.lit(device))  
    & F.col("sensor").isin(sensors)  
)
```

Στο κύριο πείραμα διατηρούνται μόνο έγκυρες μετρήσεις από το ρολόι και τους αισθητήρες `accel` και `gyro`. Το `select(...)` που ακολουθεί αφαιρεί τις βοηθητικές στήλες ελέγχου, όπως `raw_line`, `source_file`, `field_count` και `_parse_ok`, αφού δεν χρειάζονται στα επόμενα στάδια.

4.4 Οι κανόνες της εφαρμογής και ο ρόλος του Spark

Οι έλεγχοι ορίζονται από την εφαρμογή: έξι πεδία, επιτρεπτή δραστηριότητα, μη αρνητικός χρόνος και οι επιλεγμένοι αισθητήρες. Το Spark δεν αποφασίζει ποια μέτρηση είναι κατάλληλη ούτε πόσα δείγματα πρέπει να έχει ένα παράθυρο.

Αναλαμβάνει να εφαρμόσει τους ίδιους κανόνες παράλληλα στα `partitions`, να προγραμματίσει τα `tasks` και να συνδυάσει τα αποτελέσματά τους. Έτσι διαχωρίζεται η λογική του ελέγχου από τον τρόπο εκτέλεσής της.

4.5 Ποιοι καθαρισμοί δεν εφαρμόζονται

Η εφαρμογή δεν αφαιρεί ακραίες τιμές με γενικό κανόνα `z-score` ή `IQR` και δεν περιορίζει τις μεγάλες επιταχύνσεις με `winsorization`. Επίσης δεν παρεμβάλλει χαμένες μετρήσεις, δεν επαναδειγματοληπτεί σε ακριβώς 20 Hz και δεν εφαρμόζει φίλτρα χαμηλών ή υψηλών συχνοτήτων. Δεν αφαιρεί διπλές χρονικές ενδείξεις, δεν διορθώνει χειροκίνητα τις ετικέτες και δεν κανονικοποιεί τους αρχικούς άξονες κατά την ανάγνωση.

Μια μεγάλη επιτάχυνση μπορεί να είναι σφάλμα, αλλά μπορεί να προέρχεται και από κανονική κίνηση, όπως τρέξιμο, κλώτσημα ή χειροκρότημα. Χωρίς ειδικό κριτήριο για τη συγκεκριμένη δραστηριότητα, η αφαίρεσή της μπορεί να στερήσει χρήσιμη πληροφορία από το μοντέλο. Ο StandardScaler, που εφαρμόζεται αργότερα, αλλάζει την κλίμακα των χαρακτηριστικών· δεν καθαρίζει ή διαγράφει τις αρχικές μετρήσεις.

5. Χρονικά παράθυρα

5.1 Γιατί χρειάζονται πολλές διαδοχικές μετρήσεις

Ένα μεμονωμένο (x,y,z) περιγράφει μόνο μια στιγμή. Η ίδια τιμή, όπως (0, 9.8, 0), μπορεί να εμφανιστεί όταν κάποιος στέκεται, κάθεται ή περνά στιγμιαία από έναν παρόμοιο προσανατολισμό.

Η δραστηριότητα φαίνεται καλύτερα από την εξέλιξη της κίνησης: πόσο μεταβάλλονται οι τιμές, ποιο εύρος καλύπτουν και πώς συντονίζονται οι άξονες. Για αυτό πολλές διαδοχικές μετρήσεις συγκεντρώνονται σε ένα παράθυρο, το οποίο στη συνέχεια περιγράφεται με σταθερό αριθμό χαρακτηριστικών.

5.2 Πώς υπολογίζεται το χρονικό παράθυρο

Το features.py μετατρέπει τη διάρκεια του παραθύρου σε νανοδευτερόλεπτα και υπολογίζει τον αριθμό του:

```
window_ns = int(window_seconds) * 1_000_000_000
records.withColumn(
    "window_id",
    F.floor(F.col("timestamp_ns") / F.lit(window_ns)),
)
```

Ο αντίστοιχος τύπος είναι:

$$window_{id} = \left\lfloor \frac{timestamp_{ns}}{window_{seconds} \cdot 10^9} \right\rfloor$$

Στο κύριο πείραμα το window_seconds είναι 10. Με 20 μετρήσεις ανά δευτερόλεπτο αναμένονται περίπου 200 ανά αισθητήρα, ενώ διατηρούνται παράθυρα με sample_count ≥ 150.

Για τη χρονική ένδειξη 90426708196641, ο υπολογισμός δίνει:

$$\lfloor 90426708196641 / 10,000,000,000 \rfloor = 9042.$$

Οι επόμενες μετρήσεις, με διαφορά περίπου 49,5 ms, παραμένουν στο ίδιο παράθυρο μέχρι να ξεπεραστεί το επόμενο πολλαπλάσιο των 10 s. Τα όρια είναι σταθερά και τα παράθυρα δεν επικαλύπτονται.

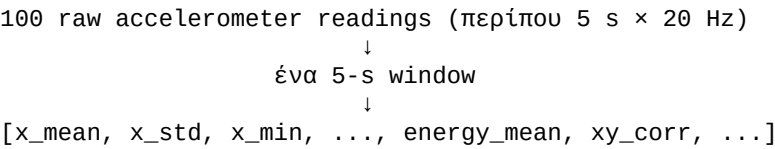
5.3 Ένα μικρό αριθμητικό παράδειγμα

Για να φανεί ο υπολογισμός, ας χρησιμοποιήσουμε τέσσερις ενδεικτικές μετρήσεις επιταχυνσιόμετρου στο ίδιο παράθυρο:

t	x	y	z	magnitude	energy
1	1	0	0	1,000	1
2	2	0	0	2,000	4
3	1	1	0	1,414	2
4	0	1	0	1,000	1

Στον άξονα x, η μέση τιμή είναι 1, το ελάχιστο 0, το μέγιστο 2 και η δειγματική τυπική απόκλιση περίπου 0,816. Η μέση ενέργεια είναι $(1+4+2+1)/4 = 2$. Στο κύριο πείραμα εφαρμόζεται η ίδια διαδικασία σε πολύ περισσότερες μετρήσεις, με ελάχιστο όριο τις 150.

Για τις ρυθμίσεις προεπεξεργασίας με παράθυρο 5 s, η ίδια ιδέα συνοψίζεται ως εξής:



5.4 Τι γίνεται όταν αλλάζει δραστηριότητα

Οι μετρήσεις ομαδοποιούνται όχι μόνο με βάση το window_id, αλλά με ολόκληρο το κλειδί: (subject_id, activity, device, sensor, window_id)

Αν η δραστηριότητα αλλάξει μέσα στο ίδιο διάστημα 10 s, δημιουργούνται δύο διαφορετικές ομάδες. Κάθε ομάδα ελέγχεται χωριστά ως προς το ελάχιστο πλήθος μετρήσεων. Έτσι, ένα μικρό τμήμα πριν ή μετά τη μετάβαση μπορεί να απορριφθεί.

Με αυτή την επιλογή, κάθε εγγραφή χαρακτηριστικών έχει μία μόνο ετικέτα δραστηριότητας. Χρησιμοποιείται όμως η ήδη γνωστή ετικέτα του WISDM, επομένως η διαδικασία δεν αξιολογεί αυτόματη ανίχνευση μεταβάσεων σε άγνωστη συνεχή ροή. Δεν χρησιμοποιούνται επικαλυπτόμενα παράθυρα ή ξεχωριστή ετικέτα μετάβασης.

6. Η διαφορά ανάμεσα σε partition και χρονικό παράθυρο

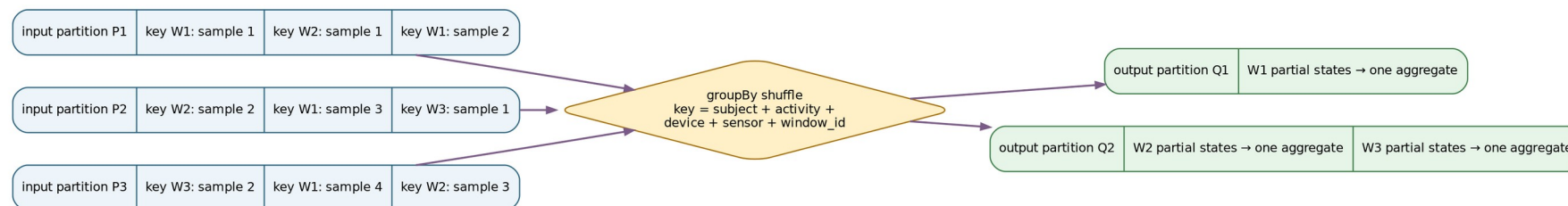
Το partition και το χρονικό παράθυρο εξυπηρετούν διαφορετικούς σκοπούς. Το πρώτο αφορά την οργάνωση του υπολογισμού· το δεύτερο την περιγραφή της κίνησης.

Έννοια	Πώς ορίζεται	Σκοπός
Spark partition	από την πηγή δεδομένων, το σχέδιο και το περιβάλλον εκτέλεσης	κατανομή δεδομένων και υπολογισμών
χρονικό παράθυρο	από το <code>floor(timestamp_ns / window_ns)</code> και το κλειδί ομαδοποίησης	περιγραφή της κίνησης σε ένα διάστημα

Οι μετρήσεις ενός παραθύρου μπορεί αρχικά να βρίσκονται σε περισσότερα από ένα partitions. Το `groupBy(...)` εξασφαλίζει ότι όλες θα συνεισφέρουν στα ίδια τελικά χαρακτηριστικά, ανεξάρτητα από τον αρχικό τους χωρισμό.

6.1 Τι κάνει το shuffle

Για να συνδυαστούν μετρήσεις με το ίδιο κλειδί, το Spark μπορεί πρώτα να υπολογίσει μερικά αποτελέσματα μέσα σε κάθε partition. Στη συνέχεια ανακατανέμει αυτά τα αποτελέσματα με βάση το κλειδί και τα συνδυάζει. Αυτή η ανακατανομή ονομάζεται **shuffle**.



Σχήμα 4. Μετρήσεις με το ίδιο κλειδί συνεισφέρουν στο ίδιο αποτέλεσμα. Μπορεί να μεταφέρονται ενδιάμεσα αθροίσματα και πλήθη αντί για όλες τις αρχικές γραμμές.

Η ανακατανομή εξασφαλίζει ότι τα αποτελέσματα θα συνδυαστούν με βάση το ίδιο κλειδί. Το κόστος της εξηγείται στην ενότητα 18 και η μέτρηση shuffle-storage του cloud στην ενότητα 21.

6.2 Πού χρειάζεται ανακατανομή στην εφαρμογή

Ανακατανομή μπορεί να χρειαστεί στην ομαδοποίηση παραθύρων, στη συνένωση επιταχυνσιομέτρου και γυροσκοπίου, στην προσθήκη του διαχωρισμού συμμετεχόντων και στις ομαδοποιημένες καταμετρήσεις. Χρησιμοποιείται επίσης σε εσωτερικούς υπολογισμούς κατά την εκπαίδευση των μοντέλων.

Δεν εκτελείται κάθε join με τον ίδιο τρόπο. Για έναν μικρό πίνακα, όπως οι 51 αντιστοιχίσεις συμμετεχόντων, το Spark μπορεί να επιλέξει broadcast αντί για ανακατανομή και των δύο πλευρών. Η εφαρμογή δεν επιβάλλει αυτή την επιλογή με ειδική υπόδειξη. Ο ακριβής τρόπος εκτέλεσης φαίνεται στο φυσικό σχέδιο της συγκεκριμένης εκτέλεσης.

7. Υπολογισμός των χαρακτηριστικών

7.1 Μέτρο και ενέργεια μιας μέτρησης

Πριν από την ομαδοποίηση, προστίθενται δύο τιμές σε κάθε μέτρηση:

```
.withColumn("magnitude", F.sqrt(F.col("x") ** 2 + F.col("y") ** 2 + F.col("z") ** 2))  
.withColumn("energy", F.col("x") ** 2 + F.col("y") ** 2 + F.col("z") ** 2)
```

Για τους τρεις άξονες (x, y, z) , οι σχέσεις είναι:

$$m = \sqrt{x^2 + y^2 + z^2}, \quad E = x^2 + y^2 + z^2 = m^2.$$

Το μέτρο περιγράφει το συνολικό μέγεθος του διανύσματος, χωρίς να εξαρτάται από το πώς κατανέμεται στους άξονες. Το τετράγωνό του δίνει μεγαλύτερο βάρος στις μεγάλες τιμές. Η μέση τιμή αυτού του τετραγώνου αποθηκεύεται ως `energy_mean`. Εδώ η λέξη «ενέργεια» δεν αναφέρεται σε φυσική ενέργεια σε joule ή σε φασματική ανάλυση με FFT.

7.2 Τα 18 χαρακτηριστικά κάθε αισθητήρα

Η λίστα `FEATURE_COLUMNS` ορίζει τα 18 χαρακτηριστικά που χρησιμοποιούνται για κάθε αισθητήρα. Χωρίζονται σε τρεις ομάδες.

A. Στατιστικά των τριών αξόνων — 12 χαρακτηριστικά

Χαρακτηριστικά	Τι περιγράφουν
<code>x_mean</code> , <code>y_mean</code> , <code>z_mean</code>	μέση στάθμη τιμών, μαζί με σταθερές συνιστώσες και προσανατολισμό
<code>x_std</code> , <code>y_std</code> , <code>z_std</code>	πόσο μεταβάλλονται οι τιμές κάθε άξονα
<code>x_min</code> , <code>y_min</code> , <code>z_min</code>	ελάχιστη τιμή στο παράθυρο
<code>x_max</code> , <code>y_max</code> , <code>z_max</code>	μέγιστη τιμή στο παράθυρο

Η διαφορά μεγίστου και ελαχίστου θα έδινε το εύρος, αλλά δεν αποθηκεύεται ως ξεχωριστή στήλη.

B. Συνολική ένταση — 3 χαρακτηριστικά

Χαρακτηριστικό	Υπολογισμός	Πληροφορία για την κίνηση
<code>magnitude_mean</code>	μέση τιμή του $\sqrt{x^2 + y^2 + z^2}$	μέση συνολική ένταση
<code>magnitude_std</code>	δειγματική τυπική απόκλιση του μέτρου	πόσο μεταβάλλεται η συνολική ένταση
<code>energy_mean</code>	μέση τιμή του $x^2 + y^2 + z^2$	μεγαλύτερη βαρύτητα στις μεγάλες τιμές

Γ. Συσχετίσεις των αξόνων — 3 χαρακτηριστικά

Χαρακτηριστικό	Έννοια
<code>xy_corr</code>	συσχέτιση Pearson μεταξύ x και y στο παράθυρο
<code>xz_corr</code>	συσχέτιση Pearson μεταξύ x και z
<code>yz_corr</code>	συσχέτιση Pearson μεταξύ y και z

Συσχέτιση κοντά στο +1 σημαίνει ότι οι τιμές των δύο αξόνων τείνουν να αυξάνονται και να μειώνονται μαζί. Κοντά στο -1 μεταβάλλονται αντίθετα, ενώ κοντά στο 0 δεν εμφανίζουν ισχυρή γραμμική σχέση. Η συσχέτιση περιγράφει τη συνύπαρξη μεταβολών, όχι την αιτία τους.

7.3 Ο υπολογισμός μέσα στο Spark

Το ακόλουθο απόσπασμα δείχνει μέρος της ομαδοποίησης και των υπολογισμών:

```
grouped = enriched.groupBy(  
    "subject_id", "activity", "device", "sensor", "window_id"  
)  
.agg(  
    F.count("*").alias("sample_count"),  
    F.mean("x").alias("x_mean"),  
    F.stddev_samp("x").alias("x_std"),  
    F.min("x").alias("x_min"),  
    F.max("x").alias("x_max"),  
    F.mean("magnitude").alias("magnitude_mean"),  
    F.mean("energy").alias("energy_mean"),  
    F.corr("x", "y").alias("xy_corr"),  
)
```

Το `groupBy` συγκεντρώνει τις μετρήσεις του ίδιου συμμετέχοντα, δραστηριότητας, συσκευής, αισθητήρα και παραθύρου. Το `count(*)` κρατά το πλήθος τους για τον έλεγχο πληρότητας· δεν αποτελεί χαρακτηριστικό του μοντέλου.

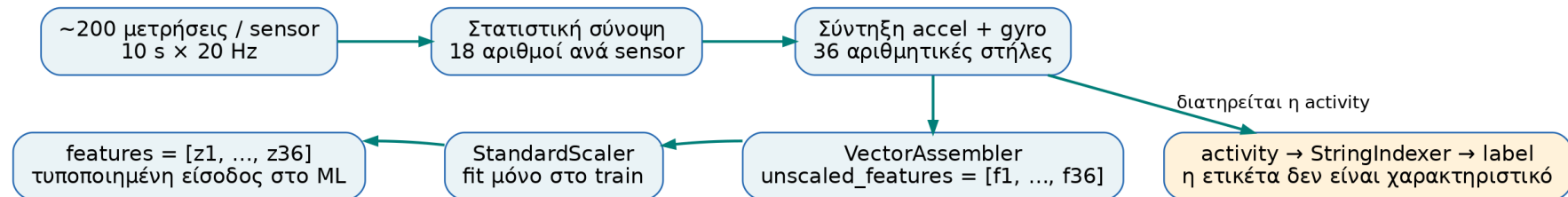
Οι `mean`, `stddev_samp`, `min` και `max` συνοψίζουν τις τιμές κάθε άξονα. Τα `magnitude` και `energy` έχουν ήδη υπολογιστεί ανά μέτρηση και τώρα συνοψίζονται ανά παράθυρο. Το `corr` υπολογίζει τη συσχέτιση δύο αξόνων, εφόσον υπάρχει η αναγκαία μεταβολή. Η πλήρης συνάρτηση επαναλαμβάνει τους υπολογισμούς για όλους τους άξονες και τα ζεύγη τους, ώστε να παραχθούν τα 18 χαρακτηριστικά.

7.4 Τι ορίζεται στον κώδικα και τι μαθαίνει το μοντέλο

Η διαδικασία μπορεί να περιγραφεί ως εξής:

raw sequence → τύποι που επέλεξε ο προγραμματιστής → 18 αριθμοί ανά sensor

Η επιλογή της μέσης τιμής, της τυπικής απόκλισης ή της συσχέτισης γίνεται στον κώδικα. Αυτά ονομάζονται χειροποίητα χαρακτηριστικά (*hand-engineered features*). Το μοντέλο δεν ανακαλύπτει τους τύπους τους· μαθαίνει πώς να αξιοποιεί τις τιμές τους για να προβλέπει τη δραστηριότητα.



Σχήμα 5. Aggregation, σύντηξη, VectorAssembler και train-fit StandardScaler. Πηγή: features.py και model.py.

8. Έλεγχος των παραθύρων και των χαρακτηριστικών

8.1 Ελάχιστο πλήθος μετρήσεων

Αμέσως μετά τον υπολογισμό των χαρακτηριστικών εφαρμόζεται το φίλτρο:

```
cleaned = grouped.where(
    F.col("sample_count") >= F.lit(int(minimum_samples))
)
```

Στο κύριο πείραμα απαιτούνται 150 μετρήσεις ανά αισθητήρα και παράθυρο. Ένα παράθυρο επιταχυνσιομέτρου με 149 μετρήσεις απορρίπτεται, ακόμη κι αν το αντίστοιχο γυροσκοπίο έχει 190. Ο έλεγχος γίνεται χωριστά για κάθε αισθητήρα, πριν από τη συνένωσή τους.

Το όριο επιτρέπει κάποια απόκλιση από τις αναμενόμενες 200 μετρήσεις χωρίς να διατηρούνται πολύ ελλιπή παράθυρα. Είναι ρύθμιση του πειράματος και όχι τιμή που επιλέγει αυτόματα το Spark.

8.2 Γιατί εμφανίζονται null ή NaN

Ορισμένα στατιστικά δεν ορίζονται για κάθε είσοδο. Η δειγματική τυπική απόκλιση, `stddev_samp`, χρειάζεται τουλάχιστον δύο μετρήσεις. Στις κύριες ρυθμίσεις προεπεξεργασίας, το όριο των 150 αποκλείει αυτή την περίπτωση, αλλά η συνάρτηση μπορεί να κληθεί και με μικρότερο όριο.

Η συσχέτιση Pearson διαιρεί με τις τυπικές αποκλίσεις των δύο αξόνων. Αν ένας άξονας παραμένει σταθερός, η συσχέτιση δεν ορίζεται και μπορεί να επιστραφεί null ή NaN. Μη έγκυρες αριθμητικές τιμές μπορούν επίσης να επηρεάσουν τα αποτελέσματα των υπολογισμών.

8.3 Αντικατάσταση των μη ορισμένων τιμών

Για κάθε χαρακτηριστικό εφαρμόζεται:

```
F.when(
    F.isnan(F.col(column)) | F.col(column).isNull(),
    F.lit(0.0),
).otherwise(F.col(column))
```

Το null ή NaN αντικαθίσταται με 0, ώστε να υπάρχει αριθμητική τιμή για το επόμενο στάδιο. Αυτό αποφεύγει αποτυχία του `VectorAssembler(handleInvalid="error")` για τις συγκεκριμένες περιπτώσεις.

Η αντικατάσταση έχει όμως ένα όριο: το μοντέλο δεν ξεχωρίζει ένα πραγματικό μηδέν από μια μη ορισμένη τιμή που αντικαταστάθηκε. Ειδικά στη συσχέτιση, το μηδέν αυτό δεν αποτελεί ένδειξη απουσίας σχέσης. Μια εναλλακτική θα ήταν πρόσθετες στήλες που δηλώνουν ότι έλειπε η τιμή, αλλά δεν εφαρμόστηκαν στην παρούσα υλοποίηση.

9. Συνένωση επιταχυνσιομέτρου και γυροσκοπίου

9.1 Τι ακριβώς ενώνεται

Η `fuse_sensor_features()` συνδυάζει τα χαρακτηριστικά όταν χρησιμοποιούνται τουλάχιστον δύο αισθητήρες. Αναζητά εγγραφές που συμφωνούν στα ακόλουθα πεδία:

```
keys = ["subject_id", "activity", "device", "window_id"]
```

Για να ξεχωρίζουν οι στήλες των δύο αισθητήρων, προστίθεται στο όνομά τους το πρόθεμα `accel_` ή `gyro_`:

```
F.col("sample_count").alias(f"{sensor}_sample_count")
```

```
F.col(column).alias(f"{sensor}_{column}")
```

Οι τελικές ονομασίες έχουν τη μορφή:

```
accel_sample_count
```

```
accel_x_mean, accel_x_std, ..., accel_energy_mean, accel_xy_corr, ...
```

```
gyro_sample_count
```

```
gyro_x_mean, gyro_x_std, ..., gyro_energy_mean, gyro_xy_corr, ...
```

9.2 Γιατί διατηρούνται μόνο τα κοινά παράθυρα

Η συνένωση γίνεται με `inner join`:

```
left.join(right, on=keys, how="inner")
```

Μια τελική εγγραφή δημιουργείται μόνο αν υπάρχουν έγκυρα χαρακτηριστικά και των δύο αισθητήρων για τον ίδιο συμμετέχοντα, δραστηριότητα, συσκευή και παράθυρο. Αν λείπει ο ένας αισθητήρας ή το παράθυρό του απορρίφθηκε από το όριο μετρήσεων, δεν δημιουργείται η συνδυασμένη εγγραφή.

Συνενώνονται οι περιγραφές ολόκληρων παραθύρων, όχι οι ακατέργαστες μετρήσεις μία προς μία. Επομένως δεν απαιτούνται ίδιες χρονικές ενδείξεις σε κάθε μέτρηση. Επίσης, το πεδίο `device` παραμένει στο κλειδί, οπότε δεν συνενώνονται τηλέφωνο και ρολόι.

9.3 Ποια χαρακτηριστικά φτάνουν στο μοντέλο

Το `model_feature_columns()` επιστρέφει τα χαρακτηριστικά με σταθερή σειρά:

```
18 accel_* columns + 18 gyro_* columns = 36 model features
```

Οι στήλες `accel_sample_count` και `gyro_sample_count` διατηρούνται για πληροφορία, αλλά δεν δίνονται στον `VectorAssembler`. Το ίδιο ισχύει για τους κωδικούς συμμετεχόντων, το `window_id`, το `subject_rank` και το `split`. Έτσι η πρόβλεψη βασίζεται στα χαρακτηριστικά της κίνησης και όχι σε αναγνωριστικά του πειράματος.

10. Τι περιγράφει μία εγγραφή του μοντέλου

Μια εγγραφή χαρακτηριστικών από έναν αισθητήρα μπορεί να έχει την ακόλουθη μορφή:

subject_id	activity	window_id	x_mean	x_std	...	energy_mean	sample_count
1600	A	9042	4,18	2,31	...	91,74	198

Οι αριθμοί είναι ενδεικτικοί και χρησιμοποιούνται μόνο για την εξήγηση της δομής. Μετά τη συνένωση αισθητήρων, οι στήλες ονομάζονται `accel_x_mean`, `gyro_x_mean` κ.ο.κ.

Η σημασία μιας εγγραφής αλλάζει καθώς προχωρά η επεξεργασία. Αρχικά περιγράφει μια στιγμή, ενώ στο μοντέλο περιγράφει ένα ολόκληρο χρονικό παράθυρο:

Στάδιο	Τι περιγράφει μία εγγραφή
αρχικό κείμενο	μία στιγμιαία μέτρηση ενός αισθητήρα
χαρακτηριστικά ανά αισθητήρα	ένα χρονικό παράθυρο ενός αισθητήρα
συνδυασμένα χαρακτηριστικά	το ίδιο παράθυρο με πληροφορία και από τους δύο αισθητήρες
είσοδος μοντέλου	ένα διάνυσμα χαρακτηριστικών X και μια ετικέτα δραστηριότητας y

11. Διάνυσμα χαρακτηριστικών, τυποποίηση και ετικέτες

11.1 Συγκέντρωση των χαρακτηριστικών σε διάνυσμα

Ο `VectorAssembler` παίρνει τις επιλεγμένες αριθμητικές στήλες και δημιουργεί μία στήλη διανυσμάτων:

```
assembler = VectorAssembler(  
    inputCols=feature_columns,  
    outputCol="unscaled_features",  
    handleInvalid="error",  
)
```

Δεν υπολογίζει νέα στατιστικά ούτε επιλέγει ποια είναι σημαντικά. Συγκεντρώνει τα ήδη υπολογισμένα χαρακτηριστικά, με τη σειρά που ορίζει το `model_feature_columns()`:

$$unscaled_features = [f_1, f_2, \dots, f_{36}]$$

Για παράδειγμα, οι παρακάτω πέντε ενδεικτικές τιμές:

x_mean	x_std	magnitude_mean	energy_mean	xy_corr
0,31	1,72	3,10	13,40	-0,20

γίνονται:

```
unscaled_features = [0.31, 1.72, 3.10, 13.40, -0.20]
```

Στο πείραμα με δύο αισθητήρες, το διάνυσμα έχει 36 θέσεις. Αυτό είναι το διάνυσμα εισόδου X που θα χρησιμοποιήσει ο ταξινομητής μετά την τυποποίηση.

11.2 StandardScaler

Τα χαρακτηριστικά δεν έχουν όλα την ίδια κλίμακα τιμών. Πριν χρησιμοποιηθούν από τη Λογιστική Παλινδρόμηση, μετατρέπονται ώστε να εκφράζουν πόσο απέχει κάθε τιμή από τον μέσο όρο της, σε μονάδες τυπικής απόκλισης.

Για παράδειγμα, αν ένα χαρακτηριστικό έχει μέση τιμή 10 και τυπική απόκλιση 2 στα δεδομένα εκπαίδευσης, η τιμή 14 μετατρέπεται σε 2: βρίσκεται δύο τυπικές αποκλίσεις πάνω από τον μέσο όρο. Πρόκειται για ενδεικτικό παράδειγμα, όχι για μέτρηση του πειράματος.

Ο μέσος όρος και η τυπική απόκλιση υπολογίζονται μόνο από τα δεδομένα εκπαίδευσης. Οι ίδιες τιμές χρησιμοποιούνται έπειτα στα δεδομένα δοκιμής. Η ρύθμιση του StandardScaler είναι:

```
StandardScaler(  
    inputCol="unscaled_features",  
    outputCol="features",  
    withMean=True,  
    withStd=True,  
)
```

Στο fit(train) υπολογίζει τη μέση τιμή και την τυπική απόκλιση κάθε χαρακτηριστικού, χρησιμοποιώντας μόνο τα δεδομένα εκπαίδευσης. Για μη μηδενική τυπική απόκλιση εφαρμόζεται:

$$z_j = \frac{x_j - \mu_{j,train}}{\sigma_{j,train}}.$$

Έτσι περιορίζεται η επίδραση της διαφορετικής κλίμακας στη Λογιστική Παλινδρόμηση. Οι ίδιες τιμές μέσου και τυπικής απόκλισης εφαρμόζονται στη δοκιμή, χωρίς να υπολογίζονται ξανά από τα νέα δεδομένα. Όταν μια διάσταση είναι σταθερή, η βιβλιοθήκη τη χειρίζεται χωρίς διαίρεση με μηδέν.

Ο scaler υπάρχει και πριν από το Random Forest, ώστε τα δύο μοντέλα να χρησιμοποιούν κοινή προετοιμασία. Τα δέντρα συνήθως δεν χρειάζονται αυτή την τυποποίηση, καθώς μια μονοτονική γραμμική αλλαγή κλίμακας δεν αλλάζει τη σειρά των τιμών τους.

11.3 Μετατροπή της δραστηριότητας σε αριθμητική ετικέτα

Η δραστηριότητα είναι η τιμή που θέλουμε να προβλέψουμε, δηλαδή το y . Ο StringIndexer μετατρέπει τον κωδικό της σε αριθμητική ετικέτα:

```
StringIndexer(  
    inputCol="activity",  
    outputCol="label",  
    handleInvalid="error",  
    stringOrderType="alphabetAsc",  
)
```

Με το stringOrderType="alphabetAsc", οι κωδικοί ταξινομούνται αλφαβητικά. Η αντιστοίχιση αποθηκεύεται στο εκπαιδευμένο στάδιο, ώστε κάθε αριθμητική πρόβλεψη να συνδέεται ξανά με τη δραστηριότητα A–S.

features		label / activity
[0.31, 1.72, ..., 13.40]	→	walking (A)

Η επιλογή handleInvalid="error" προκαλεί σφάλμα αν εμφανιστεί άγνωστη ετικέτα στη δοκιμή, αντί να την αγνοήσει. Στο συγκεκριμένο πείραμα χρησιμοποιούνται οι 18 γνωστές κλάσεις του WISDM.

12. Διαχωρισμός εκπαίδευσης, επικύρωσης και δοκιμής

12.1 Γιατί ο διαχωρισμός γίνεται ανά άνθρωπο

Κάθε άνθρωπος έχει προσωπικό ρυθμό και τρόπο κίνησης. Αν παράθυρα του ίδιου συμμετέχοντα χρησιμοποιηθούν και στην εκπαίδευση και στη δοκιμή, το μοντέλο μπορεί να αξιοποιεί χαρακτηριστικά που έχει ήδη δει. Η επίδοση τότε μπορεί να φαίνεται καλύτερη από ό,τι σε έναν νέο άνθρωπο.

Για αυτό όλα τα παράθυρα ενός συμμετέχοντα ανήκουν στο ίδιο σύνολο. Κανένα παράθυρο των ανθρώπων της δοκιμής δεν χρησιμοποιείται στο `.fit()`, ούτε για τον ταξινομητή ούτε για τον scaler και τον indexer.

12.2 Πώς προκύπτει ο σταθερός διαχωρισμός

Η `assign_subject_splits()` συλλέγει τους διαφορετικούς κωδικούς συμμετεχόντων και τους ταξινομεί με βάση το SHA-256 του συνδυασμού `seed:subject_id`:

```
subject_ids = [
    row["subject_id"]
    for row in features.select("subject_id").distinct().collect()
]
ordered = sorted(
    subject_ids,
    key=lambda subject_id:
        hashlib.sha256(f"{seed}:{subject_id}".encode()).digest(),
)
```

Στη συνέχεια υπολογίζονται τα όρια των συνόλων:

```
train_end = int(len(ordered) * 0.70)
validation_end = int(len(ordered) * (0.70 + 0.15))
```

Για 51 συμμετέχοντες, το `int(51 × 0.70)` δίνει 35 για εκπαίδευση. Το επόμενο όριο είναι `int(51 × 0.85) = 43`, οπότε οι επόμενοι 8 ανήκουν στην επικύρωση και οι τελευταίοι 8 στη δοκιμή.

Με `seed 42` και τους ίδιους κωδικούς, η σειρά παραμένει ίδια σε κάθε εκτέλεση. Ο μικρός πίνακας των 51 αντιστοιχίσεων συνενώνεται με τα χαρακτηριστικά μέσω του `subject_id`. Έτσι κάθε άνθρωπος αποκτά μία μόνο τιμή `split`, κοινή για όλα τα παράθυρά του.

12.3 Τα μεγέθη των τριών συνόλων

Στο κύριο πείραμα προέκυψαν:

Σύνολο	Συμμετέχοντες	Παράθυρα	Χρήση
εκπαίδευση	35	10.894	προσαρμογή των σταδίων και του ταξινομητή
επικύρωση	8	2.528	καταμέτρηση, χωρίς επιλογή ρυθμίσεων ή μοντέλου
δοκιμή	8	2.488	αξιολόγηση σε συμμετέχοντες εκτός εκπαίδευσης
σύνολο	51	15.910	18 δραστηριότητες

Οι ρυθμίσεις των μοντέλων είναι προκαθορισμένες. Το σύνολο επικύρωσης καταμετράται, αλλά δεν χρησιμοποιείται για επιλογή ρυθμίσεων ή μοντέλου. Οι τέσσερις συνθήκες συγκρίνονται με βάση τα αποτελέσματα στο σύνολο δοκιμής. Οι συνέπειες αυτής της επιλογής για την αξιολόγηση αναφέρονται στην ενότητα 27.2.

13. Τι λαμβάνει η εκπαίδευση του μοντέλου

Η εκπαίδευση ξεκινά με:

```
fitted = pipeline.fit(train)
```

Το `train` περιέχει 10.894 παράθυρα. Κάθε εγγραφή έχει τα 36 χαρακτηριστικά, τη δραστηριότητα και βοηθητικά πεδία όπως `subject_id`, `window_id` και `split`. Κάθε στάδιο επιλέγει μόνο τις στήλες που χρειάζεται. Ο ταξινομητής τελικά χρησιμοποιεί:

```
features: Vector[36]    label: numeric class index
```

Δεν λαμβάνει την αρχική ακολουθία των περίπου 200 μετρήσεων του παραθύρου, αλλά τη στατιστική της περιγραφή. Συνεπώς μαθαίνει σχέσεις ανάμεσα σε αυτά τα χαρακτηριστικά και τη δραστηριότητα, όχι απευθείας τη χρονική εξέλιξη των αξόνων.

Το `Pipeline.fit(train)` προσαρμόζει πρώτα τον `StringIndexer`, εφαρμόζει τον `VectorAssembler`, προσαρμόζει τον `StandardScaler` και στη συνέχεια εκπαιδεύει τον ταξινομητή. Ο `assembler` είναι μετασχηματιστής και δεν έχει δική του εκπαίδευση. Το αποτέλεσμα είναι ένα `PipelineModel` που περιέχει όλα τα προσαρμοσμένα στάδια.

14. Πολυταξική Λογιστική Παλινδρόμηση

14.1 Βασικός μηχανισμός

Το μοντέλο εξετάζει την αριθμητική περιγραφή ενός παραθύρου και δίνει ένα σκορ σε καθεμία από τις 18 δραστηριότητες. Για να το υπολογίσει, πολλαπλασιάζει κάθε χαρακτηριστικό με ένα βάρος και προσθέτει τα αποτελέσματα μαζί με μια σταθερά. Διαφορετικές δραστηριότητες έχουν διαφορετικά βάρη. Η εκπαίδευση καθορίζει τις τιμές τους από παραδείγματα με γνωστή δραστηριότητα.

Το βάρος δείχνει πώς συμβάλλει ένα χαρακτηριστικό στο σκορ μιας δραστηριότητας, όταν τα υπόλοιπα μένουν σταθερά: θετικό βάρος το αυξάνει και αρνητικό το μειώνει. Η τελική επιλογή εξαρτάται από τη σύγκριση όλων των σκορ, όχι από ένα χαρακτηριστικό μόνο. Για τη δραστηριότητα k και το διάνυσμα χαρακτηριστικών x , ο υπολογισμός γράφεται:

$$s_k = w_k^T x + b_k.$$

Τα w_k είναι τα βάρη που μαθαίνει για τη δραστηριότητα και το b_k η σταθερά της. Η softmax μετατρέπει τα σκορ σε πιθανότητες:

$$P(y=k|x) = \frac{e^{s_k}}{\sum_j e^{s_j}}.$$

Ο παρονομαστής περιλαμβάνει και τις 18 δραστηριότητες, ώστε οι πιθανότητες να αθροίζουν σε 1. Επιλέγεται η δραστηριότητα με τη μεγαλύτερη πιθανότητα.

Στην εκπαίδευση, το μοντέλο ελέγχει πόση πιθανότητα έδωσε στην πραγματική δραστηριότητα κάθε παραδείγματος. Όσο μικρότερη είναι αυτή η πιθανότητα, τόσο μεγαλύτερη είναι η ποινή. Τα βάρη αλλάζουν με στόχο να μειωθεί η συνολική ποινή, η οποία ονομάζεται λογαριθμική απώλεια (*log-loss*).

Οι ρυθμίσεις του πειράματος είναι:

```
family=multinomial
maxIter=80
regParam=0.05
elasticNetParam=0.0
```

Τα `maxIter` και `regParam` ορίζονται πριν από την εκπαίδευση. Το `elasticNetParam=0.0` επιλέγει κανονικοποίηση L2, η οποία επιβαρύνει τα μεγάλα βάρη και βοηθά στον περιορισμό της υπερπροσαρμογής, χωρίς να επιδιώκει να μηδενίσει πολλά από αυτά.

14.2 Πώς μοιράζεται ο υπολογισμός στο Spark

Όλοι οι executors συνεργάζονται για να βρεθεί ένα κοινό σύνολο βαρών. Κάθε executor εξετάζει τα παραδείγματα των partitions που επεξεργάζεται και υπολογίζει δύο ποσότητες: πόσο καλά συμφωνούν οι προβλέψεις με τις γνωστές δραστηριότητες και πώς επηρεάζεται η απώλεια από αλλαγές στα βάρη. Η δεύτερη ποσότητα είναι η κλίση (*gradient*). Χρησιμοποιείται από τον αλγόριθμο εκπαίδευσης για να υπολογίσει την επόμενη αλλαγή των βαρών.

Σε κάθε αξιολόγηση, οι τρέχοντες συντελεστές διατίθενται στους executors με broadcast, δηλαδή με αντίγραφο που μπορεί να χρησιμοποιήσει ο καθένας. Το `treeAggregate` συγκεντρώνει και συνδυάζει τις επιμέρους απώλειες και κλίσεις. Προστίθεται επίσης ο όρος L2, που επιβαρύνει τα πολύ μεγάλα βάρη. Έτσι η ενημέρωση βασίζεται στο σύνολο των δεδομένων, όχι μόνο σε ένα partition.

Για τις συγκεκριμένες ρυθμίσεις, με `elasticNetParam == 0.0` και χωρίς όρια στις παραμέτρους, το Spark 3.5.3 χρησιμοποιεί τον **LBFGS** της Breeze για την ενημέρωση των βαρών. Ο αλγόριθμος αξιοποιεί τις τρέχουσες τιμές και ένα περιορισμένο ιστορικό προηγούμενων μεταβολών. Έτσι προσεγγίζει πληροφορία για την καμπυλότητα της συνάρτησης απώλειας, χωρίς να αποθηκεύει ολόκληρο τον πίνακα δεύτερων παραγώγων.

Η διαδικασία συνεχίζεται μέχρι να ικανοποιηθεί το κριτήριο σύγκλισης ή να συμπληρωθούν οι 80 επαναλήψεις. Δεν εκπαιδεύονται ανεξάρτητες Λογιστικές Παλινδρομήσεις ανά executor· ενημερώνεται το ίδιο μοντέλο [7, 8].

15. Random Forest

15.1 Βασικός μηχανισμός

Ένα δέντρο αποφάσεων εξετάζει το παράθυρο με διαδοχικές ερωτήσεις. Μια ερώτηση μπορεί να είναι αν η μέση ενέργεια του επιταχυνσιόμετρου είναι μικρότερη από ένα όριο: $\text{accel_energy_mean} \leq \text{threshold}$. Η απάντηση καθορίζει το επόμενο κλαδί. Η διαδρομή τελειώνει σε ένα φύλλο, όπου το δέντρο εκτιμά τη δραστηριότητα.

Κατά την εκπαίδευση επιλέγονται τόσο τα χαρακτηριστικά των ερωτήσεων όσο και τα όριά τους. Το δέντρο προτιμά διαχωρισμούς που οδηγούν σε ομάδες με λιγότερο αναμειγμένες δραστηριότητες. Για παράδειγμα, μια ομάδα με κυρίως περπάτημα είναι πιο ομοιογενής από μια ομάδα με πολλές διαφορετικές δραστηριότητες σε παρόμοιες αναλογίες. Το κριτήριο Gini μετρά αυτή την ανάμειξη.

Το Random Forest δημιουργεί πολλά τέτοια δέντρα, με τυχαίες διαφοροποιήσεις στα παραδείγματα εκπαίδευσης και στα χαρακτηριστικά που εξετάζουν. Επειδή τα δέντρα δεν κάνουν πάντα τα ίδια λάθη, ο συνδυασμός τους επιδιώκει πιο σταθερές προβλέψεις. Δεν έγινε χωριστό πείραμα σύγκρισης με ένα μόνο δέντρο.

Οι υπερπαραμέτροι (hyperparameters) και το seed είναι:

```
numTrees=80  
maxDepth=12  
featureSubsetStrategy=sqrt  
seed=42
```

Το $\text{numTrees}=80$ ορίζει το πλήθος των δέντρων. Το $\text{maxDepth}=12$ περιορίζει τα διαδοχικά βήματα απόφασης μιας διαδρομής σε κάθε δέντρο. Με sqrt , σε κάθε κόμβο εξετάζεται τυχαίο υποσύνολο περίπου $\sqrt{36}=6$ χαρακτηριστικών, αντί και των 36. Το $\text{seed}=42$ καθορίζει τις τυχαίες επιλογές της εκτέλεσης.

Στην πρόβλεψη, κάθε δέντρο δίνει τις αναλογίες των δραστηριοτήτων στο φύλλο όπου καταλήγει το παράδειγμα. Σε ένα υποθετικό παράδειγμα με δύο δέντρα και δύο δραστηριότητες, το πρώτο δίνει 60% για περπάτημα και 40% για τρέξιμο, ενώ το δεύτερο δίνει 20% και 80%. Ο συνδυασμός δίνει 40% για περπάτημα και 60% για τρέξιμο, οπότε επιλέγεται το τρέξιμο. Οι αριθμοί αυτοί είναι μόνο επεξηγηματικοί, όχι αποτελέσματα του πειράματος.

Αυτόν τον συνδυασμό αναλογιών υπολογίζει η predictRaw στο Spark 3.5.3. Δεν μετρά απλώς μία ψήφο ανά δέντρο, αλλά χρησιμοποιεί την κατανομή των δραστηριοτήτων στο φύλλο. Μετά την κανονικοποίηση επιλέγεται η μεγαλύτερη πιθανότητα. Επειδή προηγείται ο StandardScaler , τα όρια των ερωτήσεων αφορούν τα τυποποιημένα χαρακτηριστικά [13].

15.2 Πώς συνεργάζονται οι executors

Για να επιλεγεί μια ερώτηση σε έναν κόμβο, χρειάζεται να εξεταστεί πώς χωρίζει τα παραδείγματα εκπαίδευσης. Τα παραδείγματα βρίσκονται σε διαφορετικά partitions, επομένως οι executors υπολογίζουν επιμέρους καταμετρήσεις ανά δραστηριότητα και πιθανό διαχωρισμό. Το Spark τις συνδυάζει πριν επιλέξει τον διαχωρισμό. Οι executors δεν κατασκευάζουν χωριστά πλήρη δάση.

Για να οργανώσει αυτούς τους υπολογισμούς, το Spark επιλέγει αρχικά υποψήφια όρια και χωρίζει τις τιμές σε διαστήματα (*bins*). Έτσι συγκεντρώνει στατιστικά ανά διάστημα αντί να εξετάζει κάθε δυνατή αριθμητική τιμή ως νέο όριο. Οι εγγραφές μετατρέπονται στην εσωτερική μορφή TreePoint και σε αναπαράσταση που υποστηρίζει την τυχαία δειγματοληψία των δέντρων (*bagging*).

Η εσωτερική αναπαράσταση διατηρείται με MEMORY_AND_DISK , επειδή χρησιμοποιείται επανειλημμένα. Κάθε partition συγκεντρώνει στατιστικά για τους κόμβους, τα διαστήματα τιμών και τις δραστηριότητες. Το reduceByKey ανακατανέμει και συνδυάζει τα στατιστικά του ίδιου κόμβου. Οι καλύτεροι διαχωρισμοί επιστρέφουν ως μικρά αποτελέσματα στον driver και ενσωματώνονται στα δέντρα.

Με επανάληψη αυτών των βημάτων αναπτύσσονται τα 80 δέντρα ενός κοινού $\text{RandomForestClassificationModel}$ [9].

16. Ένα μοντέλο με κατανεμημένους υπολογισμούς

Και στις δύο μεθόδους προκύπτει ένα μοντέλο ανά εκπαίδευση. Η κατανομή αφορά τους υπολογισμούς πάνω στα δεδομένα, όχι την επιλογή του καλύτερου από ανεξάρτητα μοντέλα των executors.

Στοιχείο	Logistic Regression	Random Forest
Δεδομένα εισόδου	διανύσματα χαρακτηριστικών σε partitions	εγγραφές οργανωμένες για τα δέντρα σε partitions
Εργασία executor	μερική απώλεια και κλίση	μερικά στατιστικά κόμβων, διαστημάτων και κλάσεων
Συνδυασμός αποτελεσμάτων	ιεραρχική συγκέντρωση με tree aggregation	ανακατανομή και συνδυασμός με reduce-by-key
Συντονισμός driver	κατάσταση βελτιστοποιητή και ενημέρωση συντελεστών	οργάνωση κόμβων και ενσωμάτωση διαχωρισμών στα δέντρα
Τελικό αποτέλεσμα	ένα πολυταξικό μοντέλο μέσα σε PipelineModel	ένα δάσος 80 δέντρων μέσα σε PipelineModel

Η εφαρμογή καλεί το Spark μέσω του Python API. Οι βασικοί υπολογισμοί των DataFrames και των μοντέλων εκτελούνται από τις υλοποιήσεις του Spark στη JVM, με τον δικό του προγραμματισμό εργασιών. Δεν μεταφέρεται κάθε εγγραφή στον Python driver για εκπαίδευση με scikit-learn.

17. Από την περιγραφή των πράξεων στην εκτέλεση

17.1 Μετασχηματισμοί και ενέργειες

Οι περισσότερες εντολές στα DataFrames περιγράφουν πώς θα προκύψει ένας νέος πίνακας. Αυτοί είναι οι μετασχηματισμοί (*transformations*). Η εκτέλεση των απαραίτητων υπολογισμών ζητείται όταν χρειάζεται ένα αποτέλεσμα, μέσω μιας ενέργειας (*action*).

Περιγραφή μετασχηματισμού	Ενέργεια που ζητά υπολογισμό
select, withColumn, where	count()
groupBy(...).agg(...)	collect()
join	.write.parquet(...) / αποθήκευση μοντέλου
.transform(test) του εκπαιδευμένου μοντέλου	καταμέτρηση προβλέψεων και υπολογισμός μετρικών
αντιστοίχιση και φιλτράρισμα συνόλων	.fit(train)

Για παράδειγμα:

```
records = valid_records(parsed, "watch", ["accel", "gyro"])
features = build_window_features(records, window_seconds=10, minimum_samples=150)
```

Οι δύο εντολές περιγράφουν την επιλογή των μετρήσεων και τον υπολογισμό χαρακτηριστικών. Δεν σημαίνουν ότι έχουν ήδη διαβαστεί όλες οι γραμμές και ολοκληρωθεί οι υπολογισμοί. Μια εντολή όπως features.count() ζητά το αποτέλεσμα και ενεργοποιεί τις εργασίες που χρειάζονται.

Αυτή είναι η βασική ιδέα της *lazy evaluation*: η μαζική επεξεργασία αναβάλλεται μέχρι να χρειαστεί. Η αναγνώριση των αρχείων και η ανάγνωση μεταδεδομένων μπορούν να γίνουν νωρίτερα, χωρίς αυτό να σημαίνει ότι έχει υπολογιστεί ολόκληρο το DataFrame.

17.2 Πώς οργανώνεται το σχέδιο εκτέλεσης

Στο προηγούμενο παράδειγμα, ο κώδικας ζητά να επιλεγούν οι έγκυρες μετρήσεις του ρολογιού και να υπολογιστούν τα χαρακτηριστικά των παραθύρων. Περιγράφει το ζητούμενο αποτέλεσμα, αλλά δεν ορίζει ποιο task θα διαβάσει κάθε τμήμα αρχείου ή πώς θα μεταφερθούν τα ενδιάμεσα αποτελέσματα. Αυτές τις αποφάσεις οργανώνει το Spark:

1. **Καταγραφή των πράξεων.** Δημιουργεί ένα λογικό σχέδιο με την ανάγνωση, τα φίλτρα και τις ομαδοποιήσεις που ζητά ο κώδικας.
2. **Έλεγχος των στηλών.** Ελέγχει, για παράδειγμα, ότι υπάρχουν οι στήλες συσκευής και χρονικής ένδειξης και ότι έχουν κατάλληλους τύπους.
3. **Βελτίωση του σχεδίου.** Ο Catalyst εξετάζει πώς μπορούν να συνδυαστούν ή να αναδιοργανωθούν πράξεις χωρίς να αλλάξει το αποτέλεσμα. Σκοπός είναι να περιοριστεί περιττή επεξεργασία.
4. **Επιλογή τρόπου εκτέλεσης.** Το φυσικό σχέδιο καθορίζει συγκεκριμένους τρόπους ανάγνωσης, ομαδοποίησης και συνένωσης. Εκεί φαίνεται και πού χρειάζεται ανακατανομή δεδομένων.
5. **Ανάθεση εργασιών.** Η εκτέλεση χωρίζεται σε στάδια και σε tasks που επεξεργάζονται partitions. Τα σημεία shuffle συνήθως χωρίζουν στάδια, επειδή οι επόμενοι υπολογισμοί χρειάζονται αποτελέσματα από προηγούμενες εργασίες.

Οι εξαρτήσεις αυτές σχηματίζουν έναν κατευθυνόμενο ακυκλικό γράφο, ή DAG: δείχνουν ποιες εργασίες πρέπει να προηγηθούν άλλων. Το lineage περιγράφει πώς προέκυψαν τα δεδομένα. Χρησιμεύει στον επανυπολογισμό τους όταν χρειαστεί, ενώ το cache επιτρέπει να επαναχρησιμοποιηθεί ένα ήδη υπολογισμένο αποτέλεσμα, όπως εξηγείται στην ενότητα 19.

18. Γιατί το shuffle έχει κόστος

Ας υποθέσουμε ότι ορισμένες μετρήσεις του παραθύρου 9042 βρίσκονται σε ένα partition και οι υπόλοιπες σε άλλο. Για να βρεθεί σωστά η μέση τιμή, πρέπει να συνδυαστούν οι πληροφορίες και από τα δύο. Δεν αρκεί πάντα ο μέσος των δύο τοπικών μέσων, επειδή τα πλήθη των μετρήσεων μπορεί να διαφέρουν.

Το Spark συνδυάζει κατάλληλα ενδιάμεσα μεγέθη, όπως άθροισμα και πλήθος, και αντίστοιχα στατιστικά για τη διακύμανση και τη συσχέτιση. Έτσι όλες οι μετρήσεις συμβάλλουν στο σωστό τελικό αποτέλεσμα.

Το shuffle απαιτεί μετατροπή των δεδομένων σε μορφή μεταφοράς, οργάνωση με ταξινόμηση ή κατακερματισμό και προσωρινή μνήμη. Μπορεί επίσης να χρησιμοποιήσει δίσκο, αν δεν χωρούν τα ενδιάμεσα δεδομένα στη μνήμη, και δίκτυο όταν οι executors βρίσκονται σε διαφορετικές μηχανές. Τα προσωρινά αρχεία παραμένουν όσο χρειάζονται για την εκτέλεση.

Το ζητούμενο δεν είναι να καταργηθεί κάθε shuffle, αλλά να αποφεύγονται οι περιττές ανακατανομές. Στην ομαδοποίηση παραθύρων είναι μέρος του μηχανισμού που συνδυάζει σωστά μετρήσεις από διαφορετικά partitions.

19. Προσωρινή μνήμη και αποθήκευση στον δίσκο

19.1 Ποια αποτελέσματα επαναχρησιμοποιούνται

Τα ίδια αποτελέσματα χρειάζονται σε περισσότερα από ένα βήματα. Για παράδειγμα, τα χαρακτηριστικά χρησιμοποιούνται τόσο για να καταμετρηθούν τα παράθυρα όσο και για να εκπαιδευτούν δύο μοντέλα. Αν δεν διατηρηθούν, μπορεί να χρειαστεί να επαναληφθεί ο υπολογισμός τους από τις αρχικές μετρήσεις.

Το cache δηλώνει ότι ένα αποτέλεσμα πρέπει να κρατηθεί για επαναχρησιμοποίηση όταν υπολογιστεί. Δεν είναι μόνιμη αποθήκευση σε αρχείο και δεν εκτελεί από μόνο του όλη την επεξεργασία. Κατά την προετοιμασία χρησιμοποιείται στα αναλυμένα δεδομένα και στα τελικά χαρακτηριστικά:

```
parsed = parse_wisdm_raw(...).cache()
```

```
features = assign_subject_splits(...).cache()
```

Κατά την εκπαίδευση διατηρούνται τα επιμέρους σύνολα και οι προβλέψεις:

```
train = features.where(split == "train").cache()
```

```
validation = features.where(split == "validation").cache()
```

```
test = features.where(split == "test").cache()
```

```
predictions = fitted.transform(test).select(...).cache()
```

Τα αναλυμένα δεδομένα χρειάζονται τόσο για τους ελέγχους ποιότητας όσο και για την επιλογή έγκυρων γραμμών. Τα χαρακτηριστικά χρησιμοποιούνται σε καταμετρήσεις και στην εκπαίδευση δύο μοντέλων. Οι προβλέψεις χρειάζονται ξανά για το πλήθος τους, τον πίνακα σύγχυσης και τις μετρικές. Το cache περιορίζει τους επανυπολογισμούς σε αυτά τα σημεία.

Το σύνολο επικύρωσης καταμετράται, αλλά δεν χρησιμοποιείται για επιλογή μοντέλου. Όταν ένα προσωρινό αποτέλεσμα δεν χρειάζεται πια, η εφαρμογή καλεί unpersist() για να αποδεσμεύσει την αποθήκευσή του.

19.2 Πού βρίσκονται τα δεδομένα ενός DataFrame

Το αντικείμενο DataFrame στον Python driver κρατά κυρίως αναφορά στο σχέδιο υπολογισμού. Δεν περιέχει υποχρεωτικά όλες τις εγγραφές του στη μνήμη του driver.

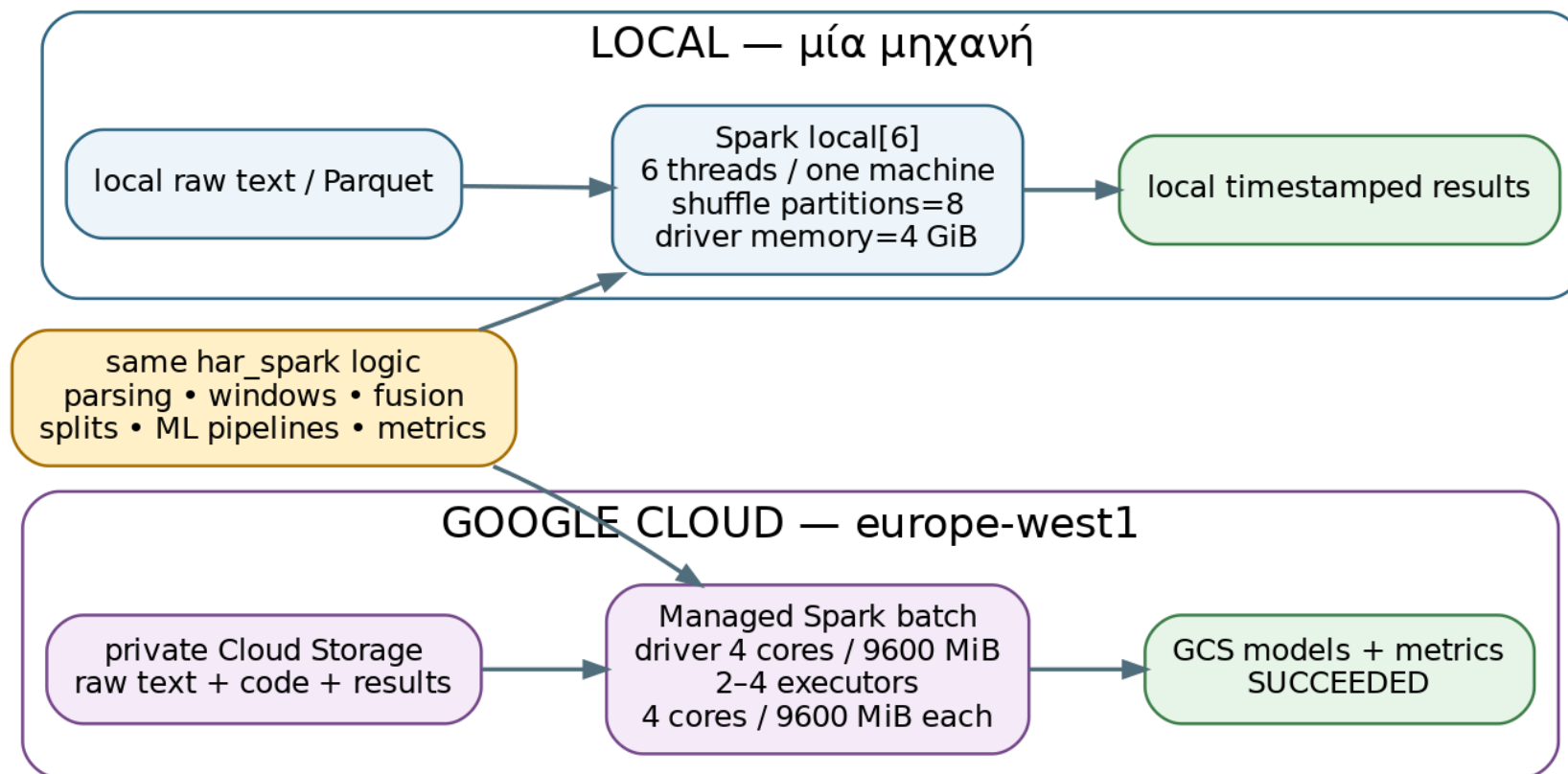
Κατά την εκτέλεση, τα partitions υπολογίζονται στους executors. Όσα διατηρούνται με cache μπορούν να χρησιμοποιηθούν ξανά, ενώ ενδιάμεσα δεδομένα του shuffle μπορεί να γραφτούν σε τοπικό δίσκο. Αν χαθεί ένα partition, το lineage επιτρέπει τον επανυπολογισμό του. Στην εργασία δεν έγινε δοκιμή αστοχίας για να μετρηθεί αυτή η διαδικασία.

Στο DataFrame API του Spark 3.5, το προεπιλεγμένο επίπεδο του cache() είναι MEMORY_AND_DISK_DESER: χρησιμοποιεί μνήμη και, για όσα δεν χωρούν, δίσκο. Αυτό διαφέρει από τη μόνιμη αποθήκευση με .write.... Τα αρχεία Parquet και τα μοντέλα παραμένουν διαθέσιμα μετά την εκτέλεση, ενώ το cache εξυπηρετεί την τρέχουσα εφαρμογή [6].

Η υλοποίηση του Random Forest διατηρεί επιπλέον την εσωτερική αναπαράσταση των δεδομένων bagging με MEMORY_AND_DISK, επειδή τη διαβάζει πολλές φορές κατά την ανάπτυξη των δέντρων.

20. Τοπική εκτέλεση και Google Cloud

Η ίδια επεξεργασία εκτελέστηκε σε δύο περιβάλλοντα. Το σχήμα συνοψίζει τις τοπικές ρυθμίσεις εκτέλεσης και τις ρυθμίσεις της καταγεγραμμένης εκτέλεσης στο Google Cloud.



Σχήμα 6. Ρυθμίσεις τοπικής και καταγεγραμμένης cloud εκτέλεσης. Πηγή: YAML και cloud/deployment.md.

20.1 Τι παραμένει ίδιο

Και στα δύο περιβάλλοντα εφαρμόζονται οι ίδιοι έλεγχοι `_parse_ok`, παράθυρα 10 s με ελάχιστο όριο 150 μετρήσεων, 18 χαρακτηριστικά ανά αισθητήρα και συνένωση των δύο αισθητήρων σε 36 χαρακτηριστικά.

Διατηρούνται επίσης ο διαχωρισμός συμμετεχόντων με SHA-256 και seed 42, η σειρά `StringIndexer` → `VectorAssembler` → `StandardScaler` → ταξινομητής, οι υπερπαραμέτροι και ο υπολογισμός των μετρικών. Η εφαρμογή του cloud εισάγει απευθείας τις κοινές συναρτήσεις των `har_spark.features`, `har_spark.ingest` και `har_spark.model._confusion_and_macro_metrics`.

20.2 Τι αλλάζει

Οι κυριότερες διαφορές αφορούν την αποθήκευση, τους διαθέσιμους πόρους και την εκκίνηση της εφαρμογής:

Στοιχείο	Τοπικά	Google Cloud
είσοδος/έξοδος	τοπικά αρχεία και Parquet	ιδιωτικό Cloud Storage
master	local[6]	περιβάλλον που παρέχει η υπηρεσία
εκτέλεση	ένα μηχάνημα, 6 νήματα	driver και 2–4 δυναμικοί executors
driver	τοπική διεργασία, ρύθμιση 4 GiB	4 πυρήνες, 9.600 MiB
executor	τοπικά νήματα εκτέλεσης	4 πυρήνες, 9.600 MiB ανά executor
shuffle partitions	τοπική ρύθμιση 8	επιλογή του περιβάλλοντος· ο κώδικας δεν επιβάλλει 8
διάθεση πόρων	ήδη διαθέσιμη μηχανή	δημιουργία πόρων για το batch
μόνιμα αποτελέσματα	σύστημα αρχείων του έργου	ξεχωριστή διαδρομή αποτελεσμάτων στο GCS
πρόσβαση	τοπικός χρήστης	service account χωρίς αρχείο κλειδιού, με περιορισμένα δικαιώματα IAM

Στη serverless εκτέλεση υπάρχουν κανονικά servers, αλλά ο χρήστης δεν διαχειρίζεται μόνιμο cluster. Η υπηρεσία διαθέτει driver και executors για το batch και αποδεσμεύει τους υπολογιστικούς πόρους όταν αυτό τερματιστεί.

20.3 Το πρόγραμμα εκκίνησης στο νέφος

Το `cloud_pipeline.py` δημιουργεί το `SparkSession` και καλεί τις κοινές συναρτήσεις προεπεξεργασίας:

```
spark = SparkSession.builder.appName("WISDM-HAR-Cloud").getOrCreate()
parsed = parse_wisdm_raw(spark, args.raw_path).cache()
records = valid_records(parsed, args.device, ["accel", "gyro"])
features = build_window_features(
    records,
    window_seconds=args.window_seconds,
    minimum_samples=args.minimum_samples,
)
features = fuse_sensor_features(features, ["accel", "gyro"])
```

Το περιβάλλον Spark παρέχεται από την υπηρεσία. Η διαδρομή των αρχείων Cloud Storage περνά ως `args.raw_path`, ενώ οι τιμές 10 s και 150 μετρήσεων δίνονται στις παραμέτρους του batch. Η ίδια συνάρτηση συνένωσης παράγει τις ίδιες στήλες χαρακτηριστικών.

Στο cloud, τα χαρακτηριστικά αποθηκεύονται πριν από την εκπαίδευση. Έτσι παραμένει διαθέσιμο ένα ενδιάμεσο αποτέλεσμα που μπορεί να ελεγχθεί ή να χρησιμοποιηθεί ξανά, χωρίς επανάληψη όλης της προεπεξεργασίας.

21. Αποτελέσματα της εκτέλεσης στο Google Cloud

Τα αποθηκευμένα στοιχεία του batch της 27-08-2026 καταγράφουν:

Πεδίο	Καταγεγραμμένη τιμή
Project	har-wisdm-student-2026
Region	europa-west1
Batch ID	har-watch-10s-20260827-055742
State	SUCCEEDED
Created	2026-08-27 05:58:00 UTC
Completed	2026-08-27 06:42:47 UTC
End-to-end duration	44 min 47 s
Runtime	2.3.39 , 2.3 LTS family
Driver	4 cores, 9.600 MiB
Executors	dynamic 2–4, 4 cores και 9.600 MiB καθένα
Usage	13,3817 DCU-hours
Shuffle-storage time	1.355,11 GB-hours
Fused windows	15.910
Split	10.894 / 2.528 / 2.488

Η υπηρεσία δίνει την υπολογιστική χρήση σε milli-DCU-seconds και τη χρήση αποθήκευσης shuffle σε GB-seconds. Οι μετατροπές σε ώρες είναι:

$48,174,200 \text{ milliDcuSeconds} / 1000 / 3600 = 13.3817 \text{ DCU-h}$
 $4,878,400 \text{ shuffleStorageGbSeconds} / 3600 = 1,355.1111 \text{ GB-h}$

Τα μεγέθη αυτά περιγράφουν την κατανάλωση της υπηρεσίας. Η χρήση shuffle-storage δεν είναι μέτρηση του όγκου δεδομένων που μεταφέρθηκε στο δίκτυο. Επίσης, δεν ισοδυναμεί από μόνη της με τελικό ποσό χρέωσης.

21.1 Μετρικές των μοντέλων στο νέφος

Model	Accuracy	Macro precision	Macro recall	Macro-F1	Weighted-F1	Train	Inference
Logistic Regression	49,96%	53,34%	49,71%	48,48%	48,63%	265,23 s	12,09 s
Random Forest	61,54%	63,80%	61,39%	61,09%	61,20%	1.911,81 s	77,81 s

Το Random Forest έδωσε υψηλότερη accuracy και macro-F1, αλλά χρειάστηκε περισσότερο χρόνο τόσο για την εκπαίδευση όσο και για την πρόβλεψη.

21.2 Σύγκριση με την τοπική εκτέλεση

Για την ίδια συνθήκη, οι τοπικές μετρήσεις ήταν:

Model	Local accuracy	Local macro-F1	Local train	Local inference
Logistic Regression	49,96%	48,48%	10,58 s	0,32 s
Random Forest	62,14%	61,90%	22,17 s	1,51 s

Η Λογιστική Παλινδρόμηση έδωσε τις ίδιες μετρικές και στα δύο περιβάλλοντα. Στο cloud, το Random Forest είχε χαμηλότερη accuracy κατά περίπου 0,60 ποσοστιαίες μονάδες και macro-F1 κατά 0,81. Μια πιθανή εξήγηση είναι η επίδραση των partitions, της σειράς των δεδομένων ή του περιβάλλοντος στην κατασκευή των δέντρων. Η συγκεκριμένη αιτία δεν ελέγχθηκε με επαναληπτικές εκτελέσεις.

Οι χρόνοι χρειάζονται διαφορετική ανάγνωση. Τα 44 min 47 s του cloud καλύπτουν όλη την εργασία, από τη διάθεση πόρων και την ανάγνωση 405,96 MB αρχικών δεδομένων έως την αποθήκευση. Οι τοπικοί χρόνοι 10,58 και 22,17 s καλύπτουν το pipeline.fit, μαζί με indexer και scaler, ξεκινώντας από έτοιμα χαρακτηριστικά Parquet. Η μέτρηση πρόβλεψης περιλαμβάνει transform, cache και count.

Άρα δεν πρόκειται για άμεση σύγκριση ταχύτητας υποδομών. Μέσα στο κύριο τοπικό πείραμα, πάντως, το Random Forest χρειάστηκε περίπου 2,1 φορές περισσότερο χρόνο εκπαίδευσης για τη βελτίωση της accuracy από 49,96% σε 62,14%. Οι χρόνοι προέρχονται από μεμονωμένες μετρήσεις.

22. Μετρικές αξιολόγησης με παραδείγματα

22.1 Ο πίνακας σύγχυσης

Ο πίνακας σύγχυσης δείχνει τις πραγματικές δραστηριότητες στις γραμμές και τις προβλέψεις στις στήλες. Η διαγώνιος περιέχει τις σωστές προβλέψεις. Ένα κελί actual=Q, predicted=F, για παράδειγμα, μετρά περιπτώσεις γραψίματος που προβλέφθηκαν ως πληκτρολόγηση.

Για να αξιολογηθεί χωριστά μια δραστηριότητα, όπως το περπάτημα, ορίζονται:

- **TP**: περπάτημα που αναγνωρίστηκε σωστά ως περπάτημα.
- **FP**: άλλη δραστηριότητα που προβλέφθηκε ως περπάτημα.
- **FN**: περπάτημα που προβλέφθηκε ως άλλη δραστηριότητα.
- **TN**: άλλη δραστηριότητα που σωστά δεν προβλέφθηκε ως περπάτημα.

22.2 Accuracy

Η accuracy είναι το ποσοστό σωστών προβλέψεων:

$$accuracy = \frac{\text{σωστές προβλέψεις}}{\text{όλες οι προβλέψεις}}.$$

Αν αναγνωριστούν σωστά 80 από 100 παράθυρα, η accuracy είναι 0,80 ή 80%. Είναι εύκολη στην ανάγνωση, αλλά επηρεάζεται περισσότερο από δραστηριότητες που έχουν πολλά παραδείγματα.

22.3 Precision, recall και F1

Το precision δείχνει πόσες προβλέψεις μιας δραστηριότητας είναι σωστές. Το recall δείχνει πόσα από τα πραγματικά παραδείγματα της εντοπίστηκαν:

$$precision = \frac{TP}{TP + FP}, \quad recall = \frac{TP}{TP + FN}.$$

Το F1 συνδυάζει τις δύο μετρικές:

$$F1 = \frac{2 \cdot precision \cdot recall}{precision + recall}.$$

Αν για το περπάτημα έχουμε TP=40, FP=10 και FN=20, τότε το precision είναι $40/50 = 0,80$, το recall $40/60 \approx 0,667$ και το F1 περίπου 0,727. Το μοντέλο είναι δηλαδή σωστό στο 80% των προβλέψεων «περπάτημα», αλλά εντοπίζει περίπου τα δύο τρίτα των πραγματικών περιπτώσεων.

22.4 Ίσο βάρος σε κάθε δραστηριότητα: macro μετρικές

Για το macro-F1 υπολογίζεται πρώτα το F1 κάθε δραστηριότητας και στη συνέχεια ο απλός μέσος των 18 τιμών:

$$macro-F1 = \frac{1}{18} \sum_{k=1}^{18} F1_k.$$

Αν υπήρχαν μόνο τρεις δραστηριότητες με F1 0,90, 0,60 και 0,30, το macro-F1 θα ήταν 0,60. Κάθε δραστηριότητα συνεισφέρει εξίσου, είτε έχει 1.000 είτε 100 παράθυρα. Έτσι η καλή επίδοση σε μια συχνή δραστηριότητα δεν καλύπτει εύκολα τη χαμηλή επίδοση σε μια σπάνια.

Με τον ίδιο τρόπο προκύπτουν τα macro precision και macro recall. Το macro-F1 δεν υπολογίζεται από αυτούς τους δύο μέσους· είναι ο μέσος των επιμέρους F1.

22.5 Weighted-F1

Το weighted-F1 υπολογίζει επίσης έναν μέσο όρο, αλλά δίνει σε κάθε δραστηριότητα βάρος ανάλογο του πλήθους των παραδειγμάτων της, του λεγόμενου *support*. Περιγράφει την επίδοση λαμβάνοντας υπόψη τη συχνότητα των κλάσεων. Το macro-F1 είναι χρήσιμο συμπλήρωμα όταν ενδιαφέρει η επίδοση σε όλες τις δραστηριότητες εξίσου.

22.6 Πώς τα υπολογίζει ο κώδικας

Ο κώδικας κρατά τις πραγματικές και τις προβλεπόμενες αριθμητικές ετικέτες και καταμετρά κάθε συνδυασμό τους:

```
predictions.select(
    F.col("label").cast("int").alias("actual_index"),
    F.col("prediction").cast("int").alias("predicted_index"),
).groupBy("actual_index", "predicted_index").count().collect()
```

Το `groupBy().count()` συνδυάζει τις 2.488 προβλέψεις του κύριου συνόλου δοκιμής σε έως 18×18 κελιά. Μόνο αυτές οι μικρές καταμετρήσεις επιστρέφουν στον `driver`. Από αυτές ο Python κώδικας υπολογίζει TP, FP, FN και τις μετρικές ανά κλάση. Οι έτοιμοι `evaluators` του Spark υπολογίζουν την `accuracy` και το `weighted-F1`. Όταν ένας λόγος δεν ορίζεται, η υλοποίηση χρησιμοποιεί τιμή μηδέν.

23. Η διαφορά από τη βαθιά μάθηση

Η εφαρμογή χρησιμοποιεί κλασική μηχανική μάθηση. Τα 36 χαρακτηριστικά των δύο αισθητήρων ορίζονται στον κώδικα. Η Λογιστική Παλινδρόμηση μαθαίνει γραμμικά βάρη και το Random Forest κανόνες δέντρων πάνω σε αυτά τα χαρακτηριστικά.

Σε ένα νευρωνικό δίκτυο που λαμβάνει χρονικές ακολουθίες, μέρος της περιγραφής της κίνησης θα μπορούσε να μαθαίνεται από τα ίδια τα δεδομένα. Η διαφορά συνοψίζεται στον πίνακα:

Παρούσα προσέγγιση	Νευρωνικό δίκτυο σε ακολουθίες
προκαθορισμένα στατιστικά mean/std/min/max/corr	εκμάθηση τοπικών χρονικών μοτίβων, π.χ. με CNN
σταθερό διάνυσμα 36 χαρακτηριστικών	ακολουθία [time, axes, sensors]
Logistic Regression / Random Forest	CNN, RNN/LSTM ή Transformer
εκπαίδευση Spark MLlib σε CPU	συχνά εκπαίδευση σε παρτίδες με GPU
άμεση ερμηνεία των χαρακτηριστικών	μαθημένες αναπαραστάσεις με λιγότερο άμεση ερμηνεία

Ένα CNN, RNN/LSTM ή Transformer θα μπορούσε να αξιοποιεί τη σειρά και τα χρονικά μοτίβα των μετρήσεων που χάνονται στις στατιστικές συνόψεις. Θα απαιτούσε όμως διαφορετική αναπαράσταση ακολουθιών, χειρισμό διαφορετικών μηκών, εκπαίδευση σε παρτίδες και περισσότερες υπερπαραμέτρους. Μπορεί να ωφελούνταν από GPU, χωρίς αυτή να αποτελεί υποχρεωτική προϋπόθεση σε κάθε περίπτωση. Στην παρούσα εφαρμογή δεν υπάρχουν νευρωνικά δίκτυα, `backpropagation` ή μαθημένες αναπαραστάσεις αυτού του τύπου.

24. PySpark ML έναντι scikit-learn

Η `scikit-learn` διαθέτει επίσης Λογιστική Παλινδρόμηση και Random Forest. Η βασική διαφορά για την εργασία είναι ο τρόπος οργάνωσης των δεδομένων και των υπολογισμών:

Στοιχείο	PySpark ML στην εφαρμογή	scikit-learn
βασική δομή	κατανεμημένο DataFrame	πίνακες στη μνήμη μιας διεργασίας
προεπεξεργασία	εκφράσεις Spark SQL, groupBy και join	pandas/NumPy ή μετασχηματιστές sklearn
χαρακτηριστικά	στήλη Spark Vector	αριθμητικός πίνακας X
αλυσίδα σταδίων	pyspark.ml.Pipeline	sklearn.pipeline.Pipeline
Logistic Regression	κατανεμημένος υπολογισμός της απώλειας	βελτιστοποίηση σε μία μηχανή με κοινή μνήμη
Random Forest	κατανεμημένα στατιστικά κόμβων	συνήθως παράλληλη εκπαίδευση δέντρων σε πυρήνες μιας μηχανής
αποθήκευση	πηγές local/HDFS/GCS	συνήθως αρχεία ή δεδομένα που φορτώνονται στη μνήμη
επέκταση πόρων	περισσότεροι executors και partitions	περισσότεροι πόροι στην ίδια μηχανή
διαχείριση εκτέλεσης	μεγαλύτερη πολυπλοκότητα	μικρότερη πολυπλοκότητα
ρόλος στην εργασία	ενιαία κατανεμημένη επεξεργασία και εκτέλεση στο νέφος	εναλλακτική για απλούστερη τοπική εκπαίδευση

Το Spark συνδέει στην ίδια εφαρμογή την κατανεμημένη ανάγνωση, την προεπεξεργασία, τη συνένωση και την εκπαίδευση. Για τις 15.910 τελικές εγγραφές, μια λύση scikit-learn σε μία μηχανή θα ήταν απλούστερη. Δεν έγινε μέτρηση της σχετικής ταχύτητας.

Η επιλογή Spark εξυπηρετεί τη μελέτη όλης της διαδρομής από τα αρχικά αρχεία έως την εκτέλεση στο νέφος. Προσφέρει επίσης δυνατότητα κατανομής της εργασίας αν τα δεδομένα αυξηθούν, χωρίς αυτό να σημαίνει ότι μετρήθηκε επιτάχυνση στο συγκεκριμένο πείραμα.

25. Παράδειγμα από τη μέτρηση έως την πρόβλεψη

Ακολουθεί η διαδρομή μιας μέτρησης του συμμετέχοντα 1600. Οι αρχικές γραμμές και οι ρυθμίσεις είναι πραγματικές, ενώ οι ενδιάμεσες τιμές χαρακτηριστικών είναι ενδεικτικές.

Βήμα 1 — Αρχικές μετρήσεις

Το αρχείο επιταχυνσιομέτρου περιέχει:

```
1600,A,90426708196641,7.091625,-0.5916671,8.195502;
1600,A,90426757696641,4.972757,-0.15831658,6.6967316;
...
```

Υπάρχει αντίστοιχο αρχείο γυροσκοπίου. Ο κωδικός A δηλώνει περπάτημα.

Βήμα 2 — Ανάλυση των πεδίων

Ο κώδικας αφαιρεί το τελικό ;, χωρίζει τα έξι πεδία και μετατρέπει τις τιμές στους κατάλληλους τύπους:

```
subject_id=1600:int, activity="A":string,  
timestamp_ns=90426708196641:long,  
device="watch", sensor="accel",  
x=7.091625:double, y=-0.5916671:double, z=8.195502:double
```

Βήμα 3 — Έλεγχος εγκυρότητας

Η γραμμή έχει έξι πεδία, γνωστή δραστηριότητα, έγκυρες αριθμητικές μετατροπές και μη αρνητικό χρόνο. Η διαδρομή αναγνωρίζεται ως επιταχυνσιόμετρο ρολογιού, οπότε το `_parse_ok` είναι `True` και η γραμμή ανήκει στα δεδομένα του κύριου πειράματος.

Βήμα 4 — Ανάγνωση σε partition

Η γραμμή διαβάζεται σε κάποιο partition. Άλλες μετρήσεις του ίδιου παραθύρου μπορεί να βρίσκονται σε διαφορετικό partition. Αυτό δεν αλλάζει τη χρονική τους θέση ή το `window_id` που θα υπολογιστεί.

Βήμα 5 — Αντιστοίχιση σε χρονικό παράθυρο

Με διάρκεια 10 s προκύπτει:

```
window_id = floor(90426708196641 / 10,000,000,000) = 9042
```

Το πλήρες κλειδί ομαδοποίησης είναι:

```
(1600, "A", "watch", "accel", 9042)
```

Βήμα 6 — Ομαδοποίηση και χαρακτηριστικά

Το `groupBy` συνδυάζει τις μετρήσεις με αυτό το κλειδί και υπολογίζει το πλήθος, τα στατιστικά των αξόνων, του μέτρου και της ενέργειας, καθώς και τις συσχετίσεις. Αν υπάρχουν, για παράδειγμα, 198 μετρήσεις, το παράθυρο περνά το όριο των 150.

Βήμα 7 — Χαρακτηριστικά γυροσκοπίου

Η ίδια διαδικασία υπολογίζει 18 χαρακτηριστικά γυροσκοπίου για το κλειδί:

```
(1600, "A", "watch", "gyro", 9042)
```

Βήμα 8 — Συνένωση αισθητήρων

Το `inner join` χρησιμοποιεί τα κοινά πεδία, χωρίς το όνομα αισθητήρα, και δημιουργεί μία εγγραφή με τις δύο ομάδες χαρακτηριστικών:

```
accel_x_mean ... accel_yz_corr  
gyro_x_mean ... gyro_yz_corr
```

Βήμα 9 — Διαχωρισμός του συμμετέχοντα

Η ταξινόμηση με SHA-256 αναθέτει όλα τα παράθυρα του συμμετέχοντα 1600 στο ίδιο σύνολο. Αν ανήκει στη δοκιμή, κανένα παράθυρό του δεν χρησιμοποιείται για προσαρμογή των σταδίων του μοντέλου.

Βήμα 10 — Διάνυσμα και ετικέτα

Ο VectorAssembler συγκεντρώνει τις 36 στήλες. Ο StandardScaler εφαρμόζει τις τιμές που έμαθε από το σύνολο εκπαίδευσης και ο StringIndexer αντιστοιχίζει τη δραστηριότητα A σε αριθμητική ετικέτα:

```
features=[z1,z2,...,z36], label=0 (αν A είναι ο αλφαριθμητικά πρώτος index)
```

Βήμα 11 — Χρήση από το μοντέλο

Αν η εγγραφή ανήκει στην εκπαίδευση, συνεισφέρει στην απώλεια και στην κλίση της Λογιστικής Παλινδρόμησης ή στα στατιστικά κόμβων του Random Forest. Αν ανήκει στη δοκιμή, το ήδη εκπαιδευμένο μοντέλο την επεξεργάζεται με `fitted.transform(test)` για να δώσει πρόβλεψη.

Βήμα 12 — Πρόβλεψη και αξιολόγηση

Η αριθμητική prediction αντιστοιχίζεται ξανά σε δραστηριότητα. Το ζεύγος πραγματικής και προβλεπόμενης δραστηριότητας αυξάνει το αντίστοιχο κελί του πίνακα σύγχυσης. Από τα αποτελέσματα και των 2.488 παραθύρων δοκιμής υπολογίζονται η accuracy και οι υπόλοιπες μετρικές.

26. Πού βρίσκεται κάθε στάδιο στον κώδικα

Ο παρακάτω πίνακας συνδέει τα στάδια με τα αρχεία και τις συναρτήσεις της εφαρμογής. Τα αποσπάσματα που ακολουθούν δείχνουν τα βασικά σημεία, όχι ολόκληρα αυτοτελή προγράμματα.

Στάδιο	Αρχείο / συνάρτηση	Λειτουργία
Ανάγνωση και έλεγχοι	ingest.py / parse_wisdm_raw	Ανάλυση πεδίων, τύποι και _parse_ok
Επιλογή δεδομένων	ingest.py / valid_records	Συσκευή και αισθητήρες του πειράματος
Παράθυρα και χαρακτηριστικά	features.py / build_window_features	Ομαδοποίηση, στατιστικά και έλεγχος πληρότητας
Συνένωση αισθητήρων	features.py / fuse_sensor_features	Κοινά παράθυρα και διακριτά ονόματα στηλών
Σειρά χαρακτηριστικών	features.py / model_feature_columns	Επιλογή 18 ή 36 στηλών
Διαχωρισμός συμμετεχόντων	features.py / assign_subject_splits	SHA-256 και μία ομάδα ανά άνθρωπο
Στάδια μοντέλου	model.py / _pipeline	Indexer, assembler, scaler και ταξινομητής
Εκπαίδευση και πρόβλεψη	model.py / train_and_evaluate	Προσαρμογή στο train και αξιολόγηση στο test
Μετρικές	model.py / _confusion_and_macro_metrics	Πίνακας σύγχυσης και macro μέσοι
Τοπική εκτέλεση	pipeline.py / prepare_features, run_from_processed	Παραγωγή ή επαναχρησιμοποίηση χαρακτηριστικών Parquet
Εκτέλεση στο νέφος	cloud/cloud_pipeline.py / main, model_pipeline	Κοινή επεξεργασία με είσοδο/έξοδο στο cloud

26.1 Έλεγχος δραστηριότητας και αριθμητικών πεδίων

```
valid_activity = F.col("activity").rlike("[A-S]$") & (F.col("activity") != "N")
required_not_null = (
    F.col("subject_id").isNotNull()
    & F.col("timestamp_ns").isNotNull()
    & F.col("x").isNotNull()
    & F.col("y").isNotNull()
    & F.col("z").isNotNull()
)
```

Η πρώτη γραμμή ορίζει τους επιτρεπτούς κωδικούς δραστηριότητας. Οι συνθήκες με & απαιτούν να μην είναι null κανένα από τα απαραίτητα αριθμητικά πεδία. Πρόκειται για εκφράσεις στηλών του Spark, όχι για επανάληψη Python πάνω σε κάθε γραμμή. Ο τελικός έλεγχος _parse_ok προσθέτει το πλήθος πεδίων, τον χρόνο και την αναγνώριση της διαδρομής.

26.2 Παράθυρα και υπολογισμός χαρακτηριστικών

```
F.floor(F.col("timestamp_ns") / F.lit(window_ns)).alias("window_id")
...
.groupBy("subject_id", "activity", "device", "sensor", "window_id")
.agg(F.count("*").alias("sample_count"), ...)
```

Το floor ορίζει σταθερά, μη επικαλυπτόμενα παράθυρα. Η δραστηριότητα στο κλειδί κρατά χωριστές τις διαφορετικές ετικέτες, ενώ το πεδίο αισθητήρα κρατά χωριστά επιταχυνσιόμετρο και γυροσκόπιο μέχρι τη συνένωσή τους. Η ομαδοποίηση συνδυάζει δεδομένα με κοινό κλειδί, ακόμη κι αν προέρχονται από διαφορετικά partitions.

26.3 Σύνδεση των σταδίων σε Pipeline

```
return Pipeline(stages=[label_indexer, assembler, scaler, classifier])
```

Ο indexer μαθαίνει την αντιστοίχιση δραστηριοτήτων σε αριθμούς, ο assembler συγκεντρώνει τα χαρακτηριστικά, ο scaler μαθαίνει τα στατιστικά τυποποίησης και ο ταξινομητής μαθαίνει βάρη ή δέντρα. Το τελικό PipelineModel εφαρμόζει τα αποθηκευμένα στάδια στα δεδομένα δοκιμής.

26.4 Προσθήκη του διαχωρισμού στα παράθυρα

```
return features.join(
    subjects.select("subject_id", "subject_rank", "split"),
    on="subject_id",
    how="inner",
)
```

Η συνένωση γίνεται μέσω του subject_id. Για αυτό όλα τα παράθυρα ενός ανθρώπου παίρνουν την ίδια τιμή split, ανεξάρτητα από το window_id τους.

26.5 Εκπαίδευση και πρόβλεψη

```
fitted = pipeline.fit(train)
predictions = fitted.transform(test).select(
    "activity", "label", "prediction"
).cache()
```

Στο fit περνά μόνο το σύνολο εκπαίδευσης. Το επιστρεφόμενο PipelineModel εφαρμόζεται στη συνέχεια στο σύνολο δοκιμής. Το select κρατά μόνο τις στήλες δραστηριότητας, πραγματικής ετικέτας και πρόβλεψης, επειδή αυτές χρειάζονται για την αξιολόγηση.

27. Επιλογές σχεδιασμού και περιορισμοί

27.1 Τι χάνεται στη στατιστική περιγραφή

Η μετατροπή μιας χρονικής ακολουθίας σε λίγα στατιστικά απλοποιεί τα δεδομένα και διευκολύνει τον κατανεμημένο υπολογισμό. Χάνει όμως μέρος της χρονικής τους δομής. Δύο δραστηριότητες μπορεί να έχουν παρόμοιους μέσους και τυπικές αποκλίσεις, αλλά διαφορετική σειρά κινήσεων. Περισσότεροι έλεγχοι για κενές τιμές δεν αποκαθιστούν αυτή την πληροφορία.

Οι παρακάτω εναλλακτικές αντιμετωπίζουν διαφορετικές πλευρές του προβλήματος:

Εναλλακτική	Πλεονέκτημα	Πρόσθετες απαιτήσεις ή περιορισμοί
χαρακτηριστικά συχνότητας ή κορυφών, π.χ. FFT	περισσότερη πληροφορία χωρίς αλλαγή της βασικής προσέγγισης Spark	νέοι υπολογισμοί χαρακτηριστικών και αξιολόγηση
επικαλυπτόμενα παράθυρα	περισσότερα παραδείγματα κοντά σε χρονικές μεταβολές	συσχετισμένα παράθυρα και μεγαλύτερος όγκος
βάρη κλάσεων και αναζήτηση ρυθμίσεων	πιθανή βελτίωση της ανάκλησης δύσκολων κλάσεων	διαδικασία επικύρωσης και περισσότερες εκπαιδεύσεις
διασταυρούμενη επικύρωση ανά συμμετέχοντα	ισχυρότερη εκτίμηση γενίκευσης	επαναλαμβανόμενη εκπαίδευση και επιπλέον κόστος
CNN/RNN/Transformer στις αρχικές ακολουθίες	εκμάθηση χρονικών μοτίβων	διαφορετική αρχιτεκτονική και προετοιμασία, ενδεχομένως GPU
scikit-learn στις 15.910 τελικές εγγραφές	απλούστερη τοπική υλοποίηση	δεν καλύπτει την ίδια κατανομημένη επεξεργασία και εκπαίδευση

Η χρήση νευρωνικών δικτύων θα άλλαζε ουσιαστικά την αναπαράσταση και την εκπαίδευση. Η παρούσα υλοποίηση αποτελεί μια βάση σύγκρισης με στατιστικά χαρακτηριστικά και Spark, πάνω στην οποία μπορούν να εξεταστούν τέτοιες επεκτάσεις.

27.2 Κύριοι περιορισμοί

- Οι 51 συμμετέχοντες και οι ελεγχόμενες συνθήκες συλλογής περιορίζουν τη γενίκευση στην καθημερινή χρήση.
- Τα σταθερά παράθυρα χάνουν μέρος της χρονικής σειράς. Τμήματα γύρω από μεταβάσεις δραστηριοτήτων μπορεί να απορριφθούν λόγω του ελάχιστου πλήθους μετρήσεων.
- Η ομαδοποίηση χρησιμοποιεί τη γνωστή δραστηριότητα, οπότε δεν αξιολογείται αυτόματη ανίχνευση άγνωστων μεταβάσεων σε συνεχή ροή.
- Η συνένωση απορρίπτει παράθυρα χωρίς έγκυρα δεδομένα και των δύο αισθητήρων. Η αντικατάσταση null/NaN με μηδέν δεν ξεχωρίζει τις ελλείπουσες από τις πραγματικά μηδενικές τιμές.
- Δεν έγινε αναζήτηση υπερπαραμέτρων. Το σύνολο επικύρωσης καταμετράται, αλλά δεν χρησιμοποιείται για επιλογή μοντέλου.
- Η συνθήκη με την υψηλότερη επίδοση επιλέγεται από τη σύγκριση στο σύνολο δοκιμής. Ανεξάρτητη επιβεβαίωση αυτής της επιλογής θα απαιτούσε νέο σύνολο δοκιμής, που δεν θα είχε χρησιμοποιηθεί στη σύγκριση.
- Η σύγκριση βασίζεται σε έναν διαχωρισμό συμμετεχόντων, χωρίς διασταυρούμενη επικύρωση ή διαστήματα εμπιστοσύνης. Οι διαφορετικές συνθήκες δεν έχουν ακριβώς τα ίδια παράθυρα δοκιμής.
- Οι χρόνοι είναι μεμονωμένες μετρήσεις, χωρίς επαναλήψεις του cloud batch. Τα όρια μέτρησης τοπικής και συνολικής cloud εκτέλεσης διαφέρουν.
- Δεν αξιολογήθηκε η καταλληλότητα για κλινικές εφαρμογές ή άλλες χρήσεις όπου ένα σφάλμα μπορεί να έχει σοβαρές συνέπειες.

28. Συμπέρασμα

Η εφαρμογή μετατρέπει τις στιγμιαίες μετρήσεις του ρολογιού σε περιγραφές χρονικών παραθύρων και στη συνέχεια σε προβλέψεις δραστηριότητας:

```
7.217.388 watch sensor measurements
→ validated typed rows
→ correctly shuffled 10-second per-sensor groups
→ 18 statistics per sensor
→ 36-feature fused windows
→ participant-disjoint X/y DataFrames
→ one distributedly fitted PipelineModel
→ predictions and class-balanced evaluation
```

Σε κάθε στάδιο αλλάζει αυτό που περιγράφει μια εγγραφή: από μία μέτρηση σε ένα παράθυρο και από εκεί σε ένα διάνυσμα χαρακτηριστικών. Ο κώδικας ορίζει τους κανόνες αυτής της μετατροπής και το Spark οργανώνει την παράλληλη εκτέλεσή τους.

Η ίδια λογική χρησιμοποιήθηκε τοπικά και στο νέφος. Οι executors συνεργάστηκαν για την εκπαίδευση κάθε μοντέλου, ενώ η δοκιμή έγινε σε χωριστούς συμμετέχοντες. Τα αποτελέσματα δείχνουν τις δυνατότητες της συγκεκριμένης προσέγγισης και ταυτόχρονα τα όριά της, ιδιαίτερα στη διάκριση παρόμοιων κινήσεων και στη μεταφορά των συμπερασμάτων σε καθημερινές συνθήκες.

Βιβλιογραφία και τεχνικές πηγές

1. G. M. Weiss, “WISDM Smartphone and Smartwatch Activity and Biometrics Dataset,” UCI Machine Learning Repository, DOI: [10.24432/C5HK59](https://doi.org/10.24432/C5HK59).
2. UCI Machine Learning Repository, [WISDM dataset record](#).
3. Τοπική επίσημη διανομή WISDM, README.txt, activity_key.txt και WISDM-dataset-description.pdf.
4. Apache Spark, [RDD Programming Guide — shuffle and persistence](#).
5. Apache Spark, [Spark SQL performance tuning and caching](#).
6. Apache Spark, [PySpark DataFrame.cache API](#).
7. Apache Spark 3.5.3 source, [LogisticRegression.scala](#).
8. Apache Spark 3.5.3 source, [RDDLossFunction.scala](#).
9. Apache Spark 3.5.3 source, [RandomForest.scala](#).
10. Google Cloud, [Serverless for Apache Spark overview](#).
11. Google Cloud, [Serverless runtime versions](#).
12. Τρέχον project evidence: cloud/deployment.md, cloud/run_summary.json, cloud/evidence/*.json και results/20260827T054435Z/*.json.
13. Apache Spark 3.5.3, [RandomForestClassificationModel.predictRaw](#).